NAZARBAYEV UNIVERSITY

Programming for Engineers

Detailed Reading Notes

Week 9: Classes, Information Hiding, and Inheritance

Yanwei Wang

School of Engineering

Nazarbayev University

Fall 2026

Define the interface. Protect the state.
Share deliberately. Specialize compatibly.

About these notes

These notes accompany the supplied 87-page presentation *Programming for Engineers: Week 9 — Classes, Information Hiding, and Inheritance*. The presentation determines the organization, terminology, engineering examples, activities, and optional scope. Short statements on the slides are developed into explanations you can study without an instructor beside you. The earlier Week 9 annotated Python companion supplies complete definitions where a slide shows only a method or an import.

Course information

Course: ENG 101 — Programming for Engineers, lecture section 2L, 6 ECTS.
Instructor: Dr. Yanwei Wang, School of Engineering, Nazarbayev University.
Term: Fall 2026.
Week 9 scope: More on classes; getters and setters; information hiding; class variables; inheritance. The source scope page identifies CLOs 1 and 5.

Lecture-slide alignment: Week 9 source PDF, pp. 1–3.

Three connected reading units

Source lecture	Central question
25: Controlled access	What can client code read or change, and what must remain true afterward?
26: Shared state	Which information belongs to one instance, and which information is deliberately shared by a class?
27: Inheritance	How can a specialized class reuse an interface without breaking its promises?

The chapters follow this sequence. Optional properties, name mangling, and string representations appear in a separate reference chapter. Worked answers follow the slide activities; additional cumulative practice and its solutions come afterward. A glossary and a page-by-page source map complete the reading companion.

How to study a state-changing program

First read the contract and predict the result. Then trace the object before and after each operation. Finally run the corresponding example file and compare your reasoning with the observed output. Do not treat a visible printed number as evidence that the expected value was returned, and do not treat an exception message as evidence that old state was preserved. Both require inspection.

Five questions for every operation

Which object is the receiver? Which name is read? Which object or attribute is changed? What result is returned? If the operation fails, which state remains?

For shared settings, record the class attribute and any instance override separately. For inheritance, identify the selected method and keep track of the *same* receiver as execution enters inherited code. These habits connect Week 4 scope, Week 5 references, Week 7 testing, and Week 8 methods to the new Week 9 material.

Code conventions and runnable files

Every Python listing contains comments; complete classes also have docstrings where helpful. Gold code panels contain Python. Grey panels contain observed output or an identified exception category. Printed line numbers belong to the notes and should not be typed into an editor.

The `examples` folder includes the supporting modules, individual demonstrations, and an `intentional_errors` subfolder.

The local file `access_models.py` supplies the `PressureReading` import. This file is supplied with the package; it is not something to install from the internet. Keep the examples together. Run a numbered file in a fresh Python process so earlier class-setting changes do not affect its answer. The separate `ENG101_Week9_Annotated_Examples.py` file is self-contained and requires only the standard library.

Different teaching versions are not one growing class definition

The deliberately leaky `ReadingSeries`, the corrected imported `ReadingSeries`, the syntax-first `TemperatureSensor`, and the later specialized class are separate examples. Do not paste them into one notebook cell sequence and assume that every earlier name still means the same class. The numbered scripts isolate these versions.

Source boundaries and worked additions

Page references refer to the **PDF page number**, including titles and dividers, not the slide's footer counter. The map at the end accounts for all 87 source pages. Complete recovered model definitions are attributed to the earlier Week 9 companion, rather than claimed to be printed in full on a particular slide. A reconstructed reporting defect on source page 72, activity solutions, extra regression checks, and cumulative practice are labelled as worked completions or extensions. The detailed adaptation record is included in the source package.

Keep three different contracts separate

Lecture 25: `PressureReading` stores finite, nonnegative *absolute* pressure in kPa.
Lecture 26: `SensorFormat` demonstrates ordinary attribute lookup and assumes a suitable numeric reading.
Lecture 27: `Sensor` and its children accept finite signed readings and ordered limits. Their unit labels and conversions do not establish full physical admissibility or a pressure reference.

An out-of-limit reading can be valid data. A method named `is_safe()` only evaluates the stated inclusive limits; it does not certify a device. Single underscores and name mangling communicate implementation intent, not secrecy. No new graded task, deadline, examination policy, or laboratory requirement is introduced by these notes.

Contents

About these notes **2**

1. Getters, Setters, and Information Hiding **6**
1.1. Warm-up: validation can be bypassed . . . 6
1.2. State an invariant first . . . 7
1.3. The public interface and stored representation . . . 8
1.4. Public, non-public, and special names . . . 8
1.5. A getter returns; a setter requests a change . . . 8
1.6. Understand the helper rather than treating it as magic . . . 9
1.7. Trace a successful update and its returned result . . . 11
1.8. Trace a rejected update . . . 12
1.9. Test both behavior and state . . . 12
1.10. Debugging: a printed value is not a returned value . . . 13
1.11. A getter can leak mutable data . . . 14
1.12. A snapshot requires a deliberate copy . . . 15
1.13. Change storage without changing the public unit . . . 16
1.14. Document the contract as a small table . . . 17
1.15. Worked activity: controlled resistance access . . . 18
1.16. Debugging: an underscore changes the attribute name . . . 19
1.17. Choose accessors for a reason . . . 20
1.18. Lecture 25 summary and exit-ticket answers . . . 20

2. Class Variables and Shared State **21**
2.1. Warm-up: decide what should be shared . . . 21
2.2. Define a class attribute and instance attributes . . . 21
2.3. Read and change a common default . . . 22
2.4. An instance can shadow an ordinary default . . . 23
2.5. Trace the applicable lookup model . . . 24
2.6. The lookup expression is a design choice . . . 24
2.7. Prediction activity: identify each binding . . . 25
2.8. A method does not use the class body as local scope . . . 26
2.9. The shared mutable-list trap . . . 26
2.10. Create independent histories in the initializer . . . 27
2.11. Mutation and rebinding lead to different structures . . . 28
2.12. A deliberate class-wide creation counter . . . 28
2.13. Why an instance increment is the wrong counter . . . 29
2.14. Say what the count does not measure . . . 30
2.15. Make shared-state tests repeatable . . . 31
2.16. Worked design clinic: ten laboratory sensors . . . 32
2.17. Lecture 26 summary and exit-ticket answers . . . 32

3. Inheritance and Specialized Engineering Classes **33**
3.1. Find shared responsibilities before writing a hierarchy . . . 33
3.2. Read the parent–child relationship . . . 33
3.3. The smallest child inherits existing behavior . . . 34
3.4. Start with the parent's public contract . . . 34
3.5. The parent validates before storing common fields . . . 35
3.6. Getters expose meaning without handing out mutable structure . . . 36

3.7. One update rule can serve the whole family . . . 37
3.8. Queries and reports still use the actual receiver . . . 38
3.9. Specialize construction using super() . . . 38
3.10. Test a conversion and the state that remains . . . 40
3.11. A pressure specialization and a compatible override . . . 40
3.12. Trace the method call and the return path . . . 41
3.13. Polymorphism: one loop, applicable behavior . . . 42
3.14. Exact type and inheritance answer different questions . . . 43
3.15. Defaults can be inherited or shadowed again . . . 43
3.16. Reusing a name is not preserving a contract . . . 44
3.17. Test inherited behavior, not just the added method . . . 45
3.18. Debugging: the parent initializer never ran . . . 46
3.19. Is-a and has-a are different relationships . . . 46
3.20. Worked activity: add a VoltageSensor . . . 47
3.21. Integrate evidence and state the limits . . . 48
3.22. Lecture 27 summary and exit-ticket answers . . . 48

4. Optional Features and Week 9 Synthesis **49**
4.1. Attribute-like access through a property . . . 49
4.2. Locate the complete numeric helper . . . 50
4.3. Name mangling changes a name, not accessibility . . . 51
4.4. Readable text and diagnostic text . . . 51
4.5. Interface and inheritance quick reference . . . 52
4.6. Suggested reading and the source basis . . . 52
4.7. Synthesis: one set of promises across three ideas . . . 53
4.8. Readiness checklist . . . 53

5. Cumulative Practice and Worked Solutions **54**
5.1. Practice questions . . . 54
5.2. Worked solutions . . . 56

6. Week 9 Glossary **59**

A. Sources, Adaptations, and Slide Coverage **62**
A.1. Direct basis and reconstruction boundaries . . . 62
A.2. Clarifications that are not silent replacements . . . 62
A.3. Page-by-page source alignment . . . 63

Bibliography **65**

1 Getters, Setters, and Information Hiding

Lecture-slide alignment: Week 9 source PDF, pp. 4–28: Lecture 25.

Week 8 introduced objects with data and methods. Week 9 asks a more demanding question: how should other parts of a program interact with that data? A useful class needs more than a place to store a number. It needs an interface that tells callers what the number means, which changes are supported, and what evidence to expect when a change is rejected.

Learning outcomes

After this chapter, you should be able to distinguish an interface from its representation, explain getters and setters, validate before changing state, interpret underscore conventions, recognize a mutable-data leak, and test the state left after an exception.

1.1. Warm-up: validation can be bypassed

Lecture-slide alignment: Week 9 source PDF, pp. 5–6.

The source warm-up creates a pressure reading with a valid initial value, then assigns a negative value directly to its public attribute. The following class definition is recovered from the Week 9 companion so that the import on the slide is not a mystery.

```
# Definition excerpt from access_models.py; imports are supplied there.
class PublicPressureReading:
    """Warm-up: a checked update can still be bypassed by direct assignment."""
    def __init__(self, value_kpa):
        self.value_kpa = checked_nonnegative(value_kpa, "Absolute pressure")

    def update(self, value_kpa):
        # The check protects this method, not all possible assignments.
        self.value_kpa = checked_nonnegative(value_kpa, "Absolute pressure")
```

Code 1.1: The warm-up class: checked construction and a checked update method

The helper rejects inappropriate numeric proposals when it is called. However, the ordinary attribute `value_kpa` is still directly assignable. Python does not infer that every later assignment must pass through a method called `update`.

```python
# ENG 101 Week 9 | Direct assignment bypasses a checked method
# Basis: slide example; source PDF page(s): 6.
from access_models import PublicPressureReading
p = PublicPressureReading(315.0)  # Constructor validates this initial kPa value.
p.value_kpa = -5.0  # Deliberate misuse: this does not call update().
print(p.value_kpa)  # A contract-violating value has been stored.
```

Code 1.2: Deliberate misuse: direct assignment bypasses the update method

```
-5.0
```

Warm-up answers

The assignment runs because `value_kpa` is an ordinary writable attribute in this class. The constructor accepted 315.0, but that earlier check does not regulate later direct assignments. The checked update method did not run. The resulting stored value violates the chosen nonnegative absolute-pressure contract.

The lesson is not that validation is useless. It is that a check protects the route on which it actually executes. An interface must specify the supported routes, and client code must use them deliberately.

1.2. State an invariant first

Lecture-slide alignment: Week 9 source PDF, p. 7.

An **invariant** is a condition promised for every valid stored state, before and after the supported operations. It is a software-model promise, not an automatic law enforced by a variable name.

Object in the slides	Promised condition
Absolute-pressure reading	Finite pressure $p \geq 0$, publicly expressed in kPa.
Ideal resistor	Finite resistance $R > 0$, expressed in ohms.
Sensor interval	Finite boundaries with lower limit no greater than upper limit.
Flat reading history	Every stored element is a finite numeric reading.

Construction must establish the promise. Each supported update must preserve it. A rejected proposal should leave the previously valid reading intact when that is the stated contract. This connects initialization, validation, documentation, and regression testing.

Engineering check

A pressure of 900 kPa can satisfy the data invariant $p \geq 0$ without satisfying a particular operating limit. The Lecture 25 class does not contain an operating-limit rule. Do not label all accepted data "safe," and do not transfer the absolute-pressure restriction to gauge-pressure data or to the generic hierarchy introduced later.

1.3. The public interface and stored representation

Lecture-slide alignment: Week 9 source PDF, p. 8.

Client code means code that uses an object. Its public interface is the documented collection of operations that the client is meant to use. The **representation** is how the object actually stores and organizes the information internally.

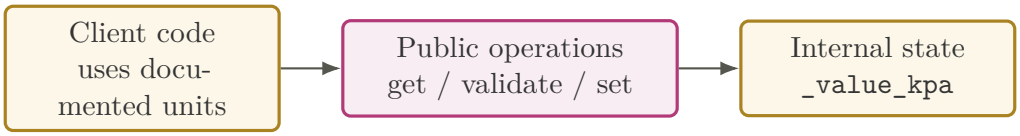


Figure 1.1.: The interface boundary from source page 8, redrawn for reading. This is a design boundary, not an access-control barrier.

Information hiding means that ordinary client code depends on the promised behavior, not on a particular backing-attribute name. A future storage change is easier when the client asks for pressure through `get_value_kpa()` instead of reaching into the object to read a storage field. Meaning must remain visible: the quantity, unit, valid domain, and failure behavior are part of the interface.

1.4. Public, non-public, and special names

Lecture-slide alignment: Week 9 source PDF, p. 9.

Name form	Meaning in this week's examples
`get_value_kpa()`	Public method intended for client use.
`_value_kpa`	Non-public implementation detail by convention.
`__value`	A double-leading-underscore name subject to name mangling, which helps avoid accidental subclass collisions.
`__init__`	A special method name defined by Python; not the same pattern as `__value`.

The single underscore is an instruction to a human reader: "ordinary client code should not depend on this." It does not make the field inaccessible. Name mangling similarly does not encrypt a value. The optional chapter demonstrates its effect directly. Neither mechanism is a security boundary.

1.5. A getter returns; a setter requests a change

Lecture-slide alignment: Week 9 source PDF, pp. 10–12.

A **getter** returns a promised quantity. A **setter** accepts a proposal and changes state only if the proposal satisfies its contract. The complete small class below joins the method and initializer excerpts displayed on the slides.

```python
# Definition excerpt from access_models.py; imports are supplied there.
class PressureReading:
    """Store one finite, nonnegative absolute pressure; public units are kPa."""
    def __init__(self, value_kpa):
        # This standalone teaching class initializes through its own setter.
        self.set_value_kpa(value_kpa)

    def get_value_kpa(self):
        """Return the current pressure in kPa without changing the object."""
        # Return the stored numeric value; leave the object unchanged.
        return self._value_kpa

    def set_value_kpa(self, proposed_kpa):
        """Accept a valid absolute pressure; on failure preserve the old value."""
        # Validate the proposal before assigning any stored attribute.
        new_value = checked_nonnegative(proposed_kpa, "Absolute pressure")
        self._value_kpa = new_value
```

Code 1.3: A complete controlled-access class recovered from the Week 9 companion

Read the class in three parts. During construction, `self` is the new instance and `value_kpa` is the caller's argument. The initializer asks the setter to establish the first valid state. In the getter, `return` sends the current number back to the caller without printing or changing it. In the setter, `proposed_kpa` is not yet trusted stored data. The checked result first receives a local name, `new_value`; only afterward is it assigned to `self._value_kpa`.

Public pressure interface

`get_value_kpa()` returns the current pressure in kPa and does not change the object. `set_value_kpa(x)` accepts a valid absolute-pressure proposal, updates the existing instance, and returns `None`. If validation raises, it does not execute the storage assignment.

The missing explicit `return` at the end of the setter is intentional. State change and returned result are separate observations. This setter is not a factory that creates and returns a replacement object.

A standalone class, not a general initialization recipe

The slides deliberately use this small class on its own. It is not the parent of the Lecture 27 hierarchy. That later parent validates local values and stores them directly rather than invoking an overridable update during base initialization. Reuse the example's stated context rather than assuming that calling a setter in every initializer is universally required.

1.6. Understand the helper rather than treating it as magic

Lecture-slide alignment: Week 9 source PDF, pp. 13 and 81.

The setter depends on a previously familiar validation function. Source page 81 prints the complete finite-number helper; it is moved here for convenience, without changing its executable logic.

```python
# Definition excerpt from numeric_checks.py; imports are supplied there.
def checked_number(value, label):
    """Return a finite float from a built-in int or float, excluding bool."""
    # A Boolean is an int subclass but not a measurement in this contract.
    if isinstance(value, bool) or not isinstance(value, (int, float)):
        raise TypeError(f"{label} must be an int or float, not bool")
    try:
        result = float(value)  # Normalize the accepted numeric representation.
    except OverflowError as error:
        raise ValueError(f"{label} is too large for a float") from error
    if not math.isfinite(result):  # Reject NaN and positive/negative infinity.
        raise ValueError(f"{label} must be finite")
    return result
```

Code 1.4: Finite-number validation: complete helper from source page 81

The first test excludes Booleans explicitly. Although `True` participates in integer relationships in Python, it is not accepted as a measurement under this contract. The next test requires an `int` or `float` instance. Numeric-looking strings such as `"315"` are not silently parsed here: input conversion belongs at a clearly identified input boundary.

The `float(value)` conversion normalizes accepted numbers to one stored numeric representation. An extremely large integer can fail conversion with `OverflowError`; the handler communicates that as a `ValueError` with a useful label. The `from error` syntax retains the original exception as context. Finally, `math.isfinite` rejects NaN and both infinities. These are explicit input checks, not assertions that can be disabled.

Precision about the type test

The source docstring describes built-in integers and floats. The implemented `isinstance` test also accepts subclasses of those types, while the separate Boolean check excludes `bool`. The code is preserved; this is a qualification of the wording, not a new numeric-type policy. The examples use ordinary Python integers and floats. See the supplementary Python reference [4].

The nonnegative helper adds one domain-specific comparison after finite-number validation. Its definition is recovered from the companion named on source page 13.

```python
# Definition excerpt from numeric_checks.py; imports are supplied there.
def checked_nonnegative(value, label):
    """Return a finite float that is at least zero."""
    result = checked_number(value, label)
    if result < 0.0:
        raise ValueError(f"{label} must be nonnegative")
    return result
```

Code 1.5: Reuse the finite check, then enforce the nonnegative data domain

```python
# ENG 101 Week 9 | Inspect helper outcomes explicitly
# Basis: worked execution of the slide contract table; source PDF page(s): 13, 81.
from numeric_checks import checked_nonnegative
for proposal in [315, 0, -5, "315", True, float("nan"), float("inf")]:
    try:
        result = checked_nonnegative(proposal, "Absolute pressure")
    except (TypeError, ValueError) as error:
        print(repr(proposal), "->", type(error).__name__)  # Rejection category.
    else:
        print(repr(proposal), "->", result)  # Normalized accepted value.
```

Code 1.6: Worked execution of the source validation table

```
315 -> 315.0
0 -> 0.0
-5 -> ValueError
'315' -> TypeError
True -> TypeError
nan -> ValueError
inf -> ValueError
```

Validation composes two ideas: “is this an acceptable numeric representation?” and “does it belong to this object’s permitted data domain?” It does not answer whether the apparatus is operating normally.

1.7. Trace a successful update and its returned result

Lecture-slide alignment: Week 9 source PDF, p. 14.

```python
# ENG 101 Week 9 | Use the public interface
# Basis: slide example; source PDF page(s): 14.
from access_models import PressureReading
p = PressureReading(315.0)
print(p.get_value_kpa())  # Query the initial value without changing it.
result = p.set_value_kpa(400.0)  # Update the same instance.
print(p.get_value_kpa())  # Query the changed state.
print(result)  # The successful setter returns None, not a new object.
```

Code 1.7: Query, update, query again, and inspect the setter’s result

```
315.0
400.0
None
```

Operation	Stored kPa	Returned result
Construct with 315.0	315.0	The class call supplies the instance.
First getter	315.0	315.0
Set to 400.0	400.0	`None`
Second getter	400.0	400.0

The object referred to by `p` is the same before and after the setter. Do not write `p = p.set_value_kpa(400.0)`: that rebinds `p` to `None`, even though the update itself succeeded. The isolated `setter_result.py` exercise in the error folder illustrates this consequence; it is a worked extension of the slide's returned-result example.

1.8. Trace a rejected update

Lecture-slide alignment: Week 9 source PDF, p. 15.

```python
# ENG 101 Week 9 | Preserve the old value after rejection
# Basis: slide example; source PDF page(s): 15.
from access_models import PressureReading
p = PressureReading(315.0)
try:
    p.set_value_kpa(-5.0)  # Reject negative absolute pressure.
except ValueError:
    print("Rejected")  # Communicate failure without inventing a reading.
print(p.get_value_kpa())  # Validation failed before the assignment.
```

Code 1.8: An exception stops the setter before the commit

```
Rejected
315.0
```

For the proposal −5 kPa, `checked_number` first produces a finite float. The nonnegative comparison then fails, so `checked_nonnegative` raises `ValueError`. Control leaves the setter before either its local checked result or its storage assignment is completed. The caller's handler prints a message. A later getter still reads 315.0.

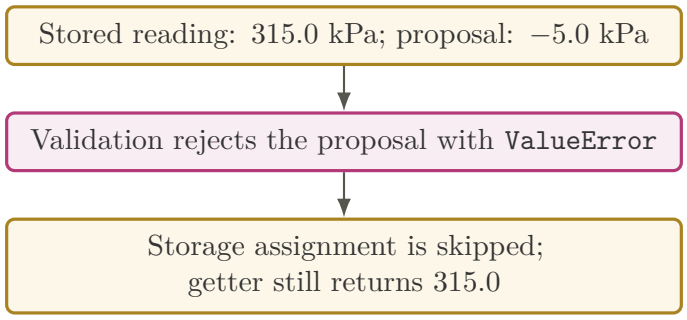


Figure 1.2.: Failure-state reasoning for source page 15. An invalid proposal is not a replacement reading.

1.9. Test both behavior and state

Lecture-slide alignment: Week 9 source PDF, pp. 16–17.

A test that merely catches an exception can miss an important defect: the setter might have stored the invalid value *before* raising. Therefore retain the previous value and check it after rejection.

Case	Expected behavior	Stored result
Initialize at 0	Accept boundary	0.0 kPa
Set 400	Accept	400.0 kPa
Set 900	Accept numeric data	900.0 kPa
Set −1	Raise `ValueError`	Previous reading
Set text	Raise `TypeError`	Previous reading
Set NaN	Raise `ValueError`	Previous reading

```python
# ENG 101 Week 9 | Test both accepted and rejected routes
# Basis: slide example; source PDF page(s): 17.
from access_models import PressureReading
p = PressureReading(0.0)  # The lower data-domain boundary is included.
assert p.get_value_kpa() == 0.0
p.set_value_kpa(900.0)  # Valid data; this class has no operating-limit rule.
assert p.get_value_kpa() == 900.0
before = p.get_value_kpa()  # Save the state before a rejected proposal.
try:
    p.set_value_kpa(-1.0)
except ValueError:
    assert p.get_value_kpa() == before  # Rejection must preserve state.
else:
    raise AssertionError("Negative absolute pressure was not rejected")
print("Access tests passed")
```

Code 1.9: A regression check for accepted boundaries and state after failure

```
Access tests passed
```

The `else` here belongs to `try`. It runs only if the setter did not raise. That is a test failure: the expected rejection was missing. Assertions express verification conditions in these ordinary runs; the model's validation remains explicit `if`/`raise` code. As in Week 7, input validity must not depend on assertions being enabled.

1.10. Debugging: a printed value is not a returned value

Lecture-slide alignment: Week 9 source PDF, p. 18.

```python
# ENG 101 Week 9 | Printing is not returning
# Basis: slide example; source PDF page(s): 18.
class Pressure:
    def __init__(self):
        self._value = 315.0
    def get_value(self):
        print(self._value)  # BUG: implicit None reaches the caller.
p = Pressure()
pressure_pa = p.get_value() * 1000.0  # Expected TypeError.
```

Code 1.10: Intentional exception: a getter prints instead of returning

```
315.0
TypeError (intentional error)
```

The printed 315.0 can give a false impression of success. The method reaches its end without a value-returning statement, so its result is `None`. The multiplication tries to multiply `None` by 1000.0 and raises `TypeError`. The failing arithmetic is a symptom; the missing returned value is the cause.

```python
# ENG 101 Week 9 | Return the value rather than printing it
# Basis: minimal repair of the slide defect; source PDF page(s): 18.
class Pressure:
    def __init__(self):
        self._value = 315.0  # kPa.
    def get_value(self):
        return self._value  # Repair: let the caller reuse the number.
p = Pressure()
pressure_pa = p.get_value() * 1000.0  # A numerical expression now works.
print(pressure_pa)
```

Code 1.11: Minimal repair: return a reusable pressure value

```
315000.0
```

The caller decides how to display the pressure. This restores composition: the getter can be used in unit conversion, tests, reports, or other calculations without capturing text printed to the terminal.

1.11. A getter can leak mutable data

Lecture-slide alignment: Week 9 source PDF, p. 19.

```python
# ENG 101 Week 9 | An internal list is exposed
# Basis: slide example; source PDF page(s): 19.
class ReadingSeries:
    def __init__(self):
        self._readings = [20.0, 21.0]  # One stored mutable list.
    def get_readings(self):
        return self._readings  # Deliberate weakness: hand out the same list.
series = ReadingSeries()
returned = series.get_readings()
returned.append(999.0)  # Mutates the list still held inside series.
print(series.get_readings())
```

Code 1.12: Deliberately weak interface: return the internal list itself

```
[20.0, 21.0, 999.0]
```

There are two access paths to one list: `series._readings` and `returned`. Calling a method whose name begins with `get` does not automatically make a copy or enforce read-only use. The client modifies the internal list using `append`, without calling any validated history-update method.

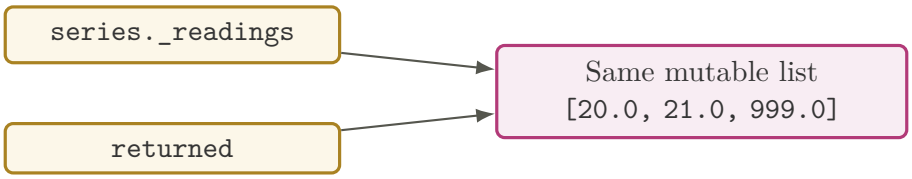


Figure 1.3.: Two references to the same list explain the unexpected internal change.

1.12. A snapshot requires a deliberate copy

Lecture-slide alignment: Week 9 source PDF, p. 20.

The corrected class is recovered from the Week 9 companion. It validates its construction input into a new outer list, then returns another new list whenever a snapshot is requested.

```python
# Definition excerpt from access_models.py; imports are supplied there.
class ReadingSeries:
    """Store a flat list of finite readings; return copies to public callers."""
    def __init__(self, readings):
        # Build a new outer list so the caller's input is not our storage.
        self._readings = [checked_number(x, "Reading") for x in readings]

    def get_readings(self):
        """Return a new list of the current numeric readings."""
        return self._readings.copy()  # Sufficient for this flat numeric list.

    def add_reading(self, value):
        # Check before modifying the stored list.
        self._readings.append(checked_number(value, "Reading"))
```

Code 1.13: A corrected flat numeric history with a copying getter

The list comprehension in the initializer is a compact version of starting with an empty list, checking each element, and appending the checked result. The `add_reading` method evaluates its argument to `append` first, so a validation failure leaves the list unchanged.

```
# ENG 101 Week 9 | Return a numeric snapshot
# Basis: slide example; source PDF page(s): 20.
from access_models import ReadingSeries
series = ReadingSeries([20.0, 21.0])
returned = series.get_readings()  # A new outer list, not internal storage.
returned.append(999.0)  # Only the caller's snapshot changes.
print(returned)
print(series.get_readings())  # The original stored readings survive.
```

Code 1.14: Changing a snapshot does not change stored readings

```
[20.0, 21.0, 999.0]
[20.0, 21.0]
```

What a shallow copy does and does not promise

The fresh outer list is enough for the flat immutable numeric elements used here. A nested list would still share its nested objects after a shallow copy. A snapshot is also not a live view: later changes to the stored series do not retroactively appear in the old copy. These are the Week 5 reference and mutation distinctions applied to an object interface.

```
# ENG 101 Week 9 | Preserve input ownership as well as returned snapshots
# Basis: supplementary state-preservation check of the supplied companion; source
    PDF page(s): 19-20.
from access_models import ReadingSeries
raw = [20.0, 21.0]
series = ReadingSeries(raw)  # Constructor validates into a new list.
raw.append(30.0)  # Changing caller input does not change the stored list.
assert series.get_readings() == [20.0, 21.0]
snapshot = series.get_readings()
series.add_reading(22.0)  # Later updates are not reflected in old snapshots.
assert snapshot == [20.0, 21.0]
print(series.get_readings())
```

Code 1.15: Additional regression check: isolate caller input and earlier snapshots

```
[20.0, 21.0, 22.0]
```

1.13. Change storage without changing the public unit

Lecture-slide alignment: Week 9 source PDF, pp. 21–22.

The first pressure class stores kPa directly. An alternative stores Pa internally while keeping kPa at the public interface. The relationships are

$$p_{\text{Pa}} = 1000 p_{\text{kPa}}, \qquad p_{\text{kPa}} = p_{\text{Pa}}/1000.$$

```python
# Definition excerpt from access_models.py; imports are supplied there.
class PressureStoredPa:
    """Same public kPa methods, different internal storage: pressure in Pa."""
    def __init__(self, value_kpa):
        self.set_value_kpa(value_kpa)

    def get_value_kpa(self):
        # Translate internal pascals back to the promised public unit.
        return self._value_pa / 1000.0

    def set_value_kpa(self, proposed_kpa):
        value_kpa = checked_nonnegative(proposed_kpa, "Absolute pressure")
        # Check the converted result too: an extreme product may overflow.
        value_pa = checked_number(value_kpa * 1000.0, "Pressure in Pa")
        self._value_pa = value_pa
```

Code 1.16: Recovered alternative representation: Pa inside, kPa at the interface

```python
# ENG 101 Week 9 | One public interface; two representations
# Basis: slide example; source PDF page(s): 22.
from access_models import PressureReading, PressureStoredPa
for model in [PressureReading, PressureStoredPa]:
    p = model(315.0)  # Both constructors receive kPa.
    p.set_value_kpa(400.0)  # Client does not inspect either storage name.
    print(p.get_value_kpa())  # Both getters promise kPa for these inputs.
```

Code 1.17: The same client uses either storage representation

```
400.0
400.0
```

The loop variable `model` refers to a class object. Calling `model(315.0)` creates an instance of the selected class. Neither run inspects the internal attribute. This is information hiding made observable: an ordinary client can keep the same operations while the implementation changes.

The compatibility claim has a numerical domain

The demonstrated values convert without difficulty. At extreme finite magnitudes, multiplying kPa by 1000 can overflow the floating representation, so the Pa-backed implementation checks the converted result and may reject a proposal that the kPa-backed class can store. The methods preserve public units, not universal identical admissible domains or bit-for-bit round trips for all floats. This qualification is recorded in the original Week 9 companion's source map.

1.14. Document the contract as a small table

Lecture-slide alignment: Week 9 source PDF, p. 23.

Operation	Promise	State change?
`get_value_kpa()`	Return the current pressure in kPa.	No
`set_value_kpa(x)`	Accept a valid proposal; return `None`.	On success
Rejected setter	Raise the appropriate exception; retain the old reading.	No
`get_readings()`	Return a fresh flat numeric snapshot.	No

A method name alone is not a specification. Document the expected input, unit, returned type, side effects, and rejection behavior. This table also guides tests and future refactoring.

1.15. Worked activity: controlled resistance access

Lecture-slide alignment: Week 9 source PDF, p. 24.

The slide asks for one ideal resistor with finite resistance $R > 0$ in ohms, a getter, a setter, and accepted and rejected tests. Zero and negative resistance must be rejected without changing an existing valid value.

Worked completion / extension

The following is a worked solution to the slide prompt. It does not introduce a new assignment. The positive-resistance rule is the stated teaching contract, not a claim that every electrical model excludes zero resistance.

```python
# Worked activity solution; imports: activity_solutions.py.
class Resistor:
    """One finite positive resistance in ohms under the activity contract."""
    def __init__(self, resistance_ohm):
        self.set_resistance_ohm(resistance_ohm)  # Establish the invariant.
    def get_resistance_ohm(self):
        # Return the quantity in the promised unit without changing state.
        return self._resistance_ohm
    def set_resistance_ohm(self, proposed_ohm):
        new_value = checked_number(proposed_ohm, "Resistance")
        if new_value <= 0.0:
            raise ValueError("Resistance must be positive")
        self._resistance_ohm = new_value  # Commit only an accepted proposal.
```

Code 1.18: Activity solution: validate finite positive resistance before assignment

```python
# ENG 101 Week 9 | Positive resistance is a different contract
# Basis: worked solution to the slide activity; source PDF page(s): 24.
from activity_solutions import Resistor
r = Resistor(4.0)
r.set_resistance_ohm(8.0)  # Accepted proposal, in ohms.
assert r.get_resistance_ohm() == 8.0
try:
    r.set_resistance_ohm(0.0)  # Zero is excluded by this particular model.
except ValueError:
    assert r.get_resistance_ohm() == 8.0  # Preserve the old valid value.
else:
    raise AssertionError("Zero resistance was not rejected")
print("Resistor tests passed")
```

Code 1.19: Accepted and rejected resistance updates

```
Resistor tests passed
```

Zero is treated differently because the two objects promise different valid domains. The pressure example includes zero absolute pressure. This resistor activity chooses strictly positive resistance, as is useful in a division-oriented ideal-resistor calculation. A method should implement its own documented domain rather than blindly copy another quantity's boundary rule.

1.16. Debugging: an underscore changes the attribute name

Lecture-slide alignment: Week 9 source PDF, p. 25.

```python
# ENG 101 Week 9 | Setter and getter use different attributes
# Basis: slide example; source PDF page(s): 25.
class Reading:
    def __init__(self):
        self._value = 20.0
    def get_value(self):
        return self._value  # Reads the non-public backing attribute.
    def set_value(self, value):
        self.value = value  # BUG: creates another ordinary attribute.
r = Reading()
r.set_value(30.0)
print(r.get_value())  # Still reads _value; no exception reveals the bug.
```

Code 1.20: Intentional logic defect: two different backing names

```
20.0
```

The getter still reads `_value`, which remains 20.0. The setter created another attribute, `value`, containing 30.0. An ordinary Python object can acquire a new attribute this way, so no exception necessarily exposes the typo. The smallest repair changes the assignment target to the same name used by the getter.

```python
# ENG 101 Week 9 | Assign the attribute read by the getter
# Basis: minimal repair; numeric input remains an assumption; source PDF page(s): 25.
class Reading:
    def __init__(self):
        self._value = 20.0
    def get_value(self):
        return self._value
    def set_value(self, value):
        self._value = value  # Minimal repair: use the same backing name.
r = Reading()
r.set_value(30.0)
assert r.get_value() == 30.0  # Retain a regression check for the typo.
print(r.get_value())
```

Code 1.21: Minimal repair and a retained set-then-get check

```
30.0
```

This repaired miniature example still assumes suitable numeric input. The repair fixes the naming defect; it does not silently add the full pressure-validation contract.

1.17. Choose accessors for a reason

Lecture-slide alignment: Week 9 source PDF, pp. 26–27.

Accessors are useful when they preserve an invariant, convert a documented internal representation, or return a snapshot instead of exposing a mutable object. They are less useful when they merely add ceremony to a simple record with no relevant rule. A value that should not be editable need not have a public setter. A query should not unexpectedly change a reading.

Before releasing a class, document public operations and units; keep implementation names out of normal client code; validate supported state-change routes; examine references returned by getters; test state after failure; and describe underscore conventions accurately. These steps matter more than how many fields have been renamed with underscores.

1.18. Lecture 25 summary and exit-ticket answers

Lecture-slide alignment: Week 9 source PDF, p. 28.

Section summary

A public interface makes units and behavior explicit while allowing representation details to change. A setter validates a proposal before committing it. Getters must be inspected for both returned meaning and reference sharing. Information hiding is a design discipline, not a security mechanism.

Exit-ticket answers: `ValueError`; the next getter returns 315.0 kPa. Validation fails before assignment, preserving the old reading. An underscore marks intent, not enforced privacy. A copying getter provides an independent flat numeric snapshot instead of exposing mutable internal storage.

2 Class Variables and Shared State

Lecture-slide alignment: Week 9 source PDF, pp. 29–53: Lecture 26.

A class can hold information that its instances share, but shared state is not automatically appropriate state. This chapter distinguishes one sensor's reading from a common reporting preference, then shows how an innocent-looking class-level list can mix independent experiments.

Learning outcomes

After this chapter, you should be able to distinguish class and instance attributes, trace ordinary lookup and shadowing, change a shared default deliberately, diagnose shared mutable data, interpret a creation counter precisely, and write repeatable tests that restore shared settings.

2.1. Warm-up: decide what should be shared

Lecture-slide alignment: Week 9 source PDF, pp. 30–31.

A current tank temperature belongs to that particular sensor. A pipe pressure belongs to a different sensor. Each experiment should normally retain its own history. In contrast, a default report precision may be one deliberate class-wide preference. A total of successful initializations is shared because its meaning is an aggregate, not because every value belongs in the class body.

Design principle

Ask what the value describes before choosing where to store it. A measurement describes an instance. A shared policy or explicitly defined aggregate may describe the class. Similar-looking assignments can therefore require different locations.

2.2. Define a class attribute and instance attributes

Lecture-slide alignment: Week 9 source PDF, pp. 32–33.

```python
# Definition excerpt from shared_models.py; imports are supplied there.
class SensorFormat:
    """An ordinary class setting provides the default report precision."""
    decimal_places = 2  # Shared default, not each sensor's measured value.

    def __init__(self, name, value):
        self.name = name
        self.value = value  # Teaching assumption: a finite numeric reading.

    def report(self):
        # Instance lookup allows an intentional per-object override.
        digits = self.decimal_places
        return f"{self.name}: {self.value:.{digits}f}"
```

Code 2.1: Recovered formatting class: a shared default and separate measurements

The assignment `decimal_places = 2` is executed in the class body and establishes an attribute on the class object. The assignments to `self.name` and `self.value` occur when the initializer runs for an instance. Their receiver is the particular object being initialized.

The `report` method first reads `self.decimal_places`. It then uses the result as a nested formatting field in `f"{self.value:.{digits}f}"`. With `digits = 2`, this requests two digits after the decimal point. The method returns text. It does not round and reassign `self.value`.

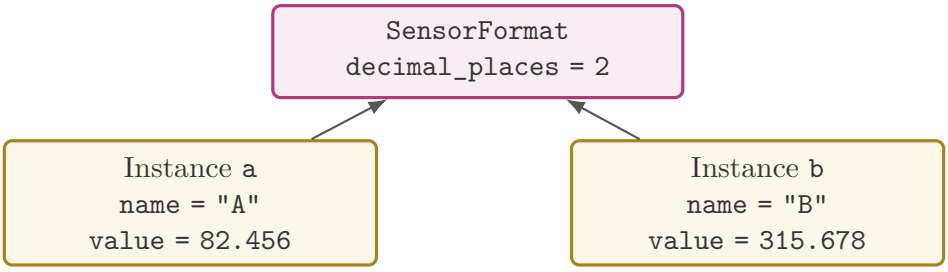


Figure 2.1.: Source page 33 redrawn: both instances can read one default while retaining separate labels and measured values.

This small class assumes suitable numeric values and formatting settings. It demonstrates lookup; it is not the more thoroughly validated sensor model in the next chapter.

2.3. Read and change a common default

Lecture-slide alignment: Week 9 source PDF, pp. 34–35.

```python
# ENG 101 Week 9 | Read the class default
# Basis: slide example; source PDF page(s): 34.
from shared_models import SensorFormat
SensorFormat.decimal_places = 2  # Controlled initial state for this run.
a = SensorFormat("A", 82.456)
b = SensorFormat("B", 315.678)
print(SensorFormat.decimal_places, a.decimal_places, b.decimal_places)
print(a.report())  # Formatting reads the shared default.
print(b.report())
```

Code 2.2: Read the class setting directly and through two instances

```
2 2 2
A: 82.46
B: 315.68
```

Neither instance has an ordinary attribute named `decimal_places`, so each read falls back to the class. That does not mean the value has been copied into each instance. The distinction becomes visible when the class attribute is changed.

```python
# ENG 101 Week 9 | Change a shared display preference
# Basis: slide example; source PDF page(s): 35.
from shared_models import SensorFormat
SensorFormat.decimal_places = 2
a = SensorFormat("A", 82.456)
b = SensorFormat("B", 315.678)
SensorFormat.decimal_places = 1  # Change display, not stored measurements.
print(a.report())
print(b.report())
```

Code 2.3: Changing the class preference affects dependent reports

```
A: 82.5
B: 315.7
```

The new reports use one decimal place. The stored values remain 82.456 and 315.678. Display precision is not measurement accuracy and is not a mutation of the measurement itself.

2.4. An instance can shadow an ordinary default

Lecture-slide alignment: Week 9 source PDF, pp. 36–37.

```python
# ENG 101 Week 9 | Instance override shadows a class default
# Basis: slide example; source PDF page(s): 36.
from shared_models import SensorFormat
SensorFormat.decimal_places = 2
a = SensorFormat("A", 82.456)
b = SensorFormat("B", 315.678)
a.decimal_places = 0  # Create an attribute on a only.
SensorFormat.decimal_places = 1  # b still reads this class setting.
print(a.decimal_places, b.decimal_places, SensorFormat.decimal_places)
print(a.report())
print(b.report())
```

Code 2.4: Create an instance override, then change the class default

```
0 1 1
A: 82
B: 315.7
```

The assignment `a.decimal_places = 0` creates an ordinary attribute on `a`; it does not assign to `SensorFormat.decimal_places`. The later class assignment therefore affects `b`, which still relies on the class, but does not replace `a`'s own override.

Operation	Class	a reads	b reads
Initial default	2	2	2
`a.decimal_places = 0`	2	0	2
`SensorFormat.decimal_places = 1`	1	0	1
`del a.decimal_places`	1	1	1

Deleting the override removes that instance binding. It does not delete the attribute from the

class, set it to zero, or restore the old class value of two. A future ordinary read finds the current class default of one.

2.5. Trace the applicable lookup model

Lecture-slide alignment: Week 9 source PDF, p. 38.

For the ordinary attributes used in the preceding examples, first ask whether the instance has its own attribute with that name. If so, use it. Otherwise, look on the class. In the single-inheritance examples that follow, the class may itself inherit the attribute.

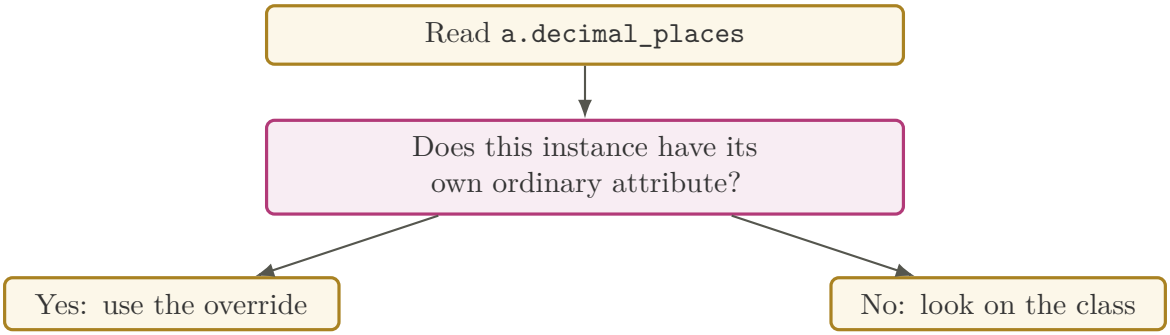


Figure 2.2.: The deliberately local lookup model on source page 38.

Technical clarification

This is not the full Python attribute-resolution algorithm. In particular, a property can intercept reading and assignment. The optional property example should not be analysed as an ordinary instance override that bypasses its setter. Advanced descriptor rules are outside this week's core scope.

2.6. The lookup expression is a design choice

Lecture-slide alignment: Week 9 source PDF, p. 39.

A report that explicitly reads `SensorFormat.decimal_places` always uses that named class's setting. A report that reads `self.decimal_places` permits an ordinary instance override and, later, a subclass default. Neither policy is universally correct: the intended flexibility determines which expression is appropriate.

The slide explicitly notes a teaching adaptation. The manuscript's original shared-formatting example uses named-class lookup; this deck also uses receiver-based lookup so the consequences of shadowing are visible.

```python
# ENG 101 Week 9 | Compare receiver lookup with named-class lookup
# Basis: worked comparison of the two displayed policies; source PDF page(s): 39.
from shared_models import SensorFormat
SensorFormat.decimal_places = 2
s = SensorFormat("A", 82.456)
s.decimal_places = 0  # Intentional local display preference.
print(s.report())  # Uses self.decimal_places and respects the override.
digits = SensorFormat.decimal_places  # Explicitly choose the named class.
print(f"{s.name}: {s.value:.{digits}f}")  # Ignores the local preference.
```

Code 2.5: Worked comparison of named-class and receiver-based lookup

```
A: 82
A: 82.46
```

The first report respects the local zero-decimal preference; the second deliberately uses the class's two-decimal policy. The measured number is unchanged in both cases.

2.7. Prediction activity: identify each binding

Lecture-slide alignment: Week 9 source PDF, p. 40.

```python
# ENG 101 Week 9 | Delete an ordinary instance override
# Basis: slide example; source PDF page(s): 40.
class Display:
    digits = 2  # One ordinary class attribute.
x = Display()
y = Display()
x.digits = 4  # Override only on x.
y.digits = 0  # Zero is a real value, not a missing attribute.
Display.digits = 1
print(x.digits, y.digits, Display.digits)
del x.digits  # Remove only x's override; leave the class setting.
print(x.digits, y.digits)
```

Code 2.6: Predict before running: two overrides and one class default

```
4 0 1
1 0
```

Answer and explanation

The first line is `4 0 1`: `x` and `y` both have overrides, while the class stores one. After deleting only `x.digits`, `x` falls back to one and `y` still reads its zero. Thus the second line is `1 0`. Zero is a stored value, not the absence of an attribute.

2.8. A method does not use the class body as local scope

Lecture-slide alignment: Week 9 source PDF, p. 41.

```python
# ENG 101 Week 9 | A bare name is not a class attribute lookup
# Basis: slide example; source PDF page(s): 41.
class Display:
    digits = 2
    def precision(self):
        return digits  # BUG: no local or global digits exists here.
print(Display().precision())  # Expected NameError.
```

Code 2.7: Intentional exception: an unqualified name is not a class lookup

```
NameError (intentional error)
```

The class attribute exists, but `return digits` does not ask for it. In this isolated example there is no local or module-global `digits` for the method to read, so the name lookup raises `NameError`. Repair the expression as `self.digits` when local overrides should be allowed, or `Display.digits` when the named class should control it.

Creating an unrelated global variable named `digits` might remove the exception while leaving the design mistaken. A correct repair makes the source of the setting explicit rather than depending on notebook execution history.

2.9. The shared mutable-list trap

Lecture-slide alignment: Week 9 source PDF, pp. 42–43.

```python
# ENG 101 Week 9 | Independent experiments share one list
# Basis: slide example; source PDF page(s): 42.
class Experiment:
    readings = []  # BUG for independent histories: allocated once here.
a = Experiment()
b = Experiment()
a.readings.append(10.0)  # Look up and mutate the class list.
b.readings.append(20.0)  # This is still the very same list.
print(a.readings)
print(b.readings)
print(a.readings is b.readings)  # Identity, not element equality.
```

Code 2.8: Intentional design defect: separate experiments use one class list

```
[10.0, 20.0]
[10.0, 20.0]
True
```

The class body evaluates `[]` once when that class is defined. Both instances then read the same list through the class attribute. Calling `append` mutates the selected list; it does not assign a fresh attribute on either instance. Therefore the two additions become one combined history.

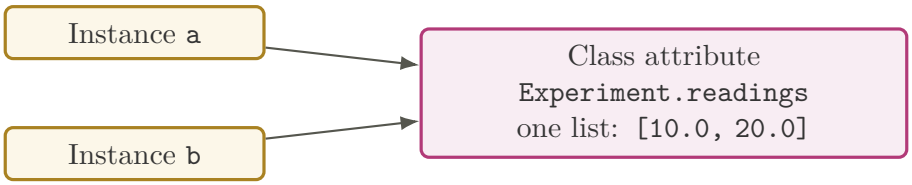


Figure 2.3.: The source page 43 ownership problem: two experiment instances do not imply two history lists.

Engineering check

Mixing experiments this way destroys provenance without an explicit data-processing step. The mistake concerns ownership of mutable storage, not whether two constructor calls occurred. Separate class instances can still refer to shared objects.

2.10. Create independent histories in the initializer

Lecture-slide alignment: Week 9 source PDF, pp. 44–45.

```python
# Definition excerpt from shared_models.py; imports are supplied there.
class IndependentHistory:
    """Each instance owns a separate list; input values are finite numbers."""
    def __init__(self, name):
        self.name = name
        self.history = []  # A fresh list is created during every call.

    def add(self, value):
        self.history.append(checked_number(value, "Reading"))
```

Code 2.9: A fresh list is allocated for each instance

```python
# ENG 101 Week 9 | Each instance owns its history
# Basis: slide example; source PDF page(s): 45.
from shared_models import IndependentHistory
a = IndependentHistory("A")  # Initialization creates a fresh list.
b = IndependentHistory("B")
a.add(10.0)
b.add(20.0)
print(a.history)
print(b.history)
print(a.history is b.history)  # Distinct list objects.
```

Code 2.10: Observe both histories and compare their identities

```
[10.0]
[20.0]
False
```

Every call to `__init__` evaluates `[]` again, giving that instance a separate outer list. The comparison with `is` asks whether the two names refer to the same object. The comparison with `==` would instead ask whether the current elements are equal. Two newly created empty histories can satisfy equality while failing identity.

The public `history` field still permits deliberate external mutation. This class fixes independent ownership and validates additions made through `add`; it is not claimed to enforce the snapshot interface of `ReadingSeries`. Distinguish the responsibility being demonstrated from a stronger unimplemented promise.

2.11. Mutation and rebinding lead to different structures

Lecture-slide alignment: Week 9 source PDF, p. 46.

```
# ENG 101 Week 9 | Trace mutation and rebinding separately
# Basis: slide example; source PDF page(s): 46.
class Store:
    readings = []  # An intentionally shared class list for this trace.
a = Store()
b = Store()
a.readings.append(10.0)  # Mutate the class list; no instance binding yet.
b.readings = [20.0]  # A new list and a new ordinary attribute on b.
a.readings.append(30.0)  # Still changes the class list.
print(Store.readings)
print(a.readings)
print(b.readings)
```

Code 2.11: Trace the class list and the instance list separately

```
[10.0, 30.0]
[10.0, 30.0]
[20.0]
```

After the operation	Class list / a reads	b reads
Create both instances	`[]`	Same class list
`a.readings.append(10.0)`	`[10.0]`	Same class list
`b.readings = [20.0]`	`[10.0]`	Own `[20.0]`
`a.readings.append(30.0)`	`[10.0, 30.0]`	Own `[20.0]`

The assignment to `b.readings` shadows the class attribute with a new list. It does not remove elements from the old list or change `a`'s lookup. For an actual independent-history design, allocate each list in the initializer instead of relying on a later accidental shadowing operation.

2.12. A deliberate class-wide creation counter

Lecture-slide alignment: Week 9 source PDF, pp. 47–48.

```python
# Definition excerpt from shared_models.py; imports are supplied there.
class CountedReading:
    """Count successful initializations of this standalone class in this run."""
    total_created = 0

    def __init__(self, value):
        # A rejected input must not increase the successful-creation count.
        new_value = checked_number(value, "Reading")
        self.value = new_value
        CountedReading.total_created += 1
```

Code 2.12: Count successful initializations after validation succeeds

The counter is incremented only after a finite checked value has been obtained and stored. Its name is accessed through `CountedReading`, so the class-wide binding is updated rather than a new per-instance binding being created.

```python
# ENG 101 Week 9 | Count only successful initializations
# Basis: slide example; source PDF page(s): 48.
from shared_models import CountedReading
CountedReading.total_created = 0  # Fresh fixture for this isolated example.
a = CountedReading(10.0)
b = CountedReading(20.0)
try:
    CountedReading("bad")  # Rejected before the count is incremented.
except TypeError:
    print("Invalid creation rejected")
print(CountedReading.total_created)
```

Code 2.13: A failed proposal does not count as a successful initialization

```
Invalid creation rejected
2
```

The initial assignment of zero sets up a controlled demonstration. In a multi-component application, resetting a shared counter would affect the meaning of the aggregate; it should not occur casually during normal operation.

2.13. Why an instance increment is the wrong counter

Lecture-slide alignment: Week 9 source PDF, p. 49.

```python
# ENG 101 Week 9 | An instance increment is not a class total
# Basis: slide example; source PDF page(s): 49.
class Counter:
    total = 0
    def __init__(self):
        self.total += 1  # BUG: read class zero, then bind instance one.
a = Counter()
b = Counter()
print(a.total, b.total, Counter.total)  # Two local ones; shared zero.
```

Code 2.14: Intentional logic defect: augmented assignment creates a local override

```
1 1 0
```

For this ordinary integer attribute, `self.total += 1` reads the value found through the instance, computes a new integer, and assigns that result back to the instance. The first object reads the class zero and stores its own one. The second does the same. Neither operation changes `Counter.total`, which remains zero.

```python
# ENG 101 Week 9 | Update the intended shared counter
# Basis: minimal repair of the slide counter defect; source PDF page(s): 49.
class Counter:
    total = 0
    def __init__(self):
        Counter.total += 1  # Change the class binding, not an instance.
a = Counter()
b = Counter()
print(a.total, b.total, Counter.total)  # Both fall back to the shared total.
```

Code 2.15: Minimal repair: explicitly update the named class attribute

```
2 2 2
```

This explanation is about an integer and ordinary attribute assignment. It should not be generalized to say that every augmented assignment on every mutable object creates a fresh object. The earlier list examples are a reminder to inspect the operation and its operand type.

2.14. Say what the count does not measure

Lecture-slide alignment: Week 9 source PDF, p. 50.

Proposed interpretation	Meaning of this counter
Successful initializations in this run	Yes, for this standalone implementation.
Currently live objects	No. Removing a name does not decrement it.
Attempted creations including failures	No. Invalid proposals fail earlier.
Number of readings or updates	No. Updates do not increment it.
Permanent identifiers across runs or computers	No. It is an in-memory aggregate.

Technical clarification

The counter is designed for the standalone `CountedReading` class. If a parent initializer counted itself and a child failed later, that would not automatically count successful complete child constructions. Object-lifetime tracking, concurrent updates, and durable identifiers are different designs and are not part of this exercise.

2.15. Make shared-state tests repeatable

Lecture-slide alignment: Week 9 source PDF, p. 51.

```python
# ENG 101 Week 9 | Test equal values and distinct identities
# Basis: slide example; source PDF page(s): 51.
from shared_models import IndependentHistory
a = IndependentHistory("A")
b = IndependentHistory("B")
a.add(10.0)  # Deliberately change only one instance.
assert a.history == [10.0]
assert b.history == []  # Other experiment stays unchanged.
assert a.history is not b.history
print("Independent history tests passed")
```

Code 2.16: A state change in one instance must not leak into another

```
Independent history tests passed
```

Test independent histories by changing one and checking the other, not merely by comparing two initial empty lists. For class settings, start from a known fixture and restore changed state afterward. Otherwise one successful test can change the answer of a later test.

Worked completion / extension

The following implements the source slide's restoration guidance. The `finally` clause is the Week 7 cleanup idea: restore the saved shared setting whether the assertion succeeds or an exception interrupts the test. It is not additional core class syntax.

```python
# ENG 101 Week 9 | Save and restore a shared test fixture
# Basis: worked implementation of the slide testing guidance; source PDF page(s): 51.
from shared_models import SensorFormat
original_digits = SensorFormat.decimal_places  # Save shared state.
try:
    SensorFormat.decimal_places = 1  # Fixture for this small test.
    s = SensorFormat("A", 82.456)
    assert s.report() == "A: 82.5"
finally:
    SensorFormat.decimal_places = original_digits  # Also runs after failure.
assert SensorFormat.decimal_places == original_digits
print("Shared setting restored")
```

Code 2.17: Save and restore a class-wide test setting

```
Shared setting restored
```

A fresh process also isolates demonstrations. When using notebooks, restarting and running all cells in order prevents a hidden earlier assignment from controlling a later result.

2.16. Worked design clinic: ten laboratory sensors

Lecture-slide alignment: Week 9 source PDF, p. 52.

Names, units, readings, and per-sensor histories belong to the individual objects under this design. A common default number of report digits can be class-level, as can an explicitly documented aggregate creation count.

Storing histories or current measurements only at class level can mix or overwrite information from different sensors. Storing the supposed aggregate counter separately in every instance prevents it from aggregating. Storing a common formatting preference separately can be appropriate for individual preferences, but changing one instance would not then update the others. The consequences, rather than the length of the code, justify the choice.

2.17. Lecture 26 summary and exit-ticket answers

Lecture-slide alignment: Week 9 source PDF, p. 53.

Section summary

Shared defaults are not automatically copied into every instance. An ordinary instance assignment can shadow a class attribute. Mutating a shared object differs from rebinding an instance attribute. A count is meaningful only when its increment rule and limits are stated precisely.

In the exit ticket, `a.digits = 4` establishes an override, and the later class setting becomes one. Thus `a` reads four while `b` reads one. A new per-experiment list should be created during each initialization. Appending to a class-level list changes an existing shared object; assigning a new instance list changes where that instance's future ordinary lookup finds its value.

3 Inheritance and Specialized Engineering Classes

Lecture-slide alignment: Week 9 source PDF, pp. 54–78: Lecture 27.

A temperature sensor and a pressure sensor share several responsibilities: each has a label, a unit, a current reading, an operating rule, a checked update, and a report. They also have differences. Inheritance can express that relationship without duplicating every common method. Reuse is useful only when the specialized object still respects the interface expected by its callers.

Learning outcomes

After this chapter, you should be able to identify a parent–child relationship, inherit and initialize common behavior, add or override a method, trace method selection, process different sensor types through one interface, test inherited guarantees, and distinguish an *is-a* relationship from a *has-a* relationship.

3.1. Find shared responsibilities before writing a hierarchy

Lecture-slide alignment: Week 9 source PDF, pp. 55–56.

The source warm-up separates common operations from conversions specific to a quantity. Labels, finite readings, ordered limits, checked updates, and text reports can be common. Celsius-to-Kelvin, kPa-to-Pa, and V-to-mV conversions have different meanings and belong to appropriate specializations.

Design principle

A common method name is not enough. The shared operation needs a common meaning: its input contract, returned type, unit convention, and state-change policy must be compatible across the objects that use it.

3.2. Read the parent–child relationship

Lecture-slide alignment: Week 9 source PDF, p. 57.

The terms **parent**, **base class**, and **superclass** describe the more general class. The terms **child**, **derived class**, and **subclass** describe a specialization. In this example, a `TemperatureSensor` is a `Sensor`; it is not a container holding a separate hidden parent sensor.

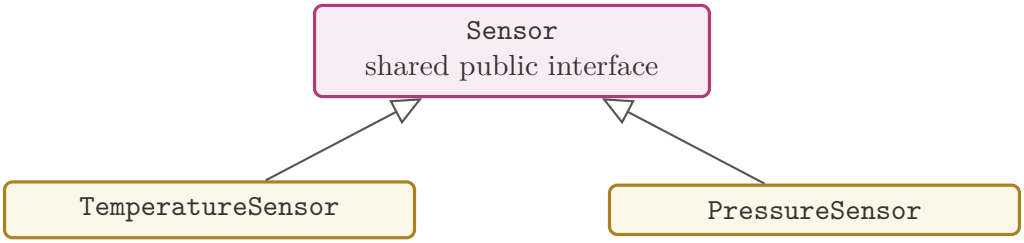


Figure 3.1.: The single-inheritance hierarchy from source page 57. Hollow arrows point toward the general parent.

This chapter stays with a small, single-inheritance hierarchy. The goal is to understand initialization, receiver identity, reuse, and contracts, not to maximize the number of classes.

3.3. The smallest child inherits existing behavior

Lecture-slide alignment: Week 9 source PDF, p. 58.

```python
# ENG 101 Week 9 | Inherit an initializer unchanged
# Basis: slide example; source PDF page(s): 58.
from sensor_hierarchy import Sensor
class TemperatureSensor(Sensor):
    pass  # No new initializer: parent interface still requires the unit.
s = TemperatureSensor("Tank", "deg C", 82.5, 0.0, 90.0)
print(s.report())  # Uses the inherited reporting method.
print(isinstance(s, Sensor))
```

Code 3.1: A syntax-first child with no replacement initializer

```
Tank: 82.5 deg C | WITHIN LIMITS
True
```

The `pass` statement adds no behavior. The inherited initializer still expects a name, unit, value, lower limit, and upper limit. A name such as `TemperatureSensor` does not automatically choose Celsius. In this first version the caller still supplies `"deg C"` explicitly.

The same object can use inherited getters, update methods, the limit query, and the report. Its exact class is the local child class even though the implementation of the initializer and report is found on the parent. The complete parent used by this import is explained next and is included in `sensor_hierarchy.py`.

Do not mix two versions of TemperatureSensor

The local syntax-first class above has a five-argument initialization interface, including the unit. The later imported specialization fixes the unit and uses four caller arguments. They are deliberately isolated demonstrations, not simultaneous definitions to merge into one notebook without care.

3.4. Start with the parent's public contract

Lecture-slide alignment: Week 9 source PDF, p. 59.

Public operation	Meaning in this hierarchy
`get_name()`, `get_unit()`	Return documented labels. A label does not perform a conversion.
`get_value()`, `get_limits()`	Return the current reading and the inclusive limit pair.
`set_value(x)`, `update(x)`	Accept a finite numeric reading; retain old state on rejection.
`is_safe()`	Return a Boolean for compliance with the supplied inclusive limits.
`report()`	Return report text without changing the reading.

Not the absolute-pressure-only model

This generic hierarchy accepts finite *signed* readings and ordered finite limits. The Lecture 25 `PressureReading` class instead requires a nonnegative absolute pressure. Do not silently impose that rule on this parent or its pressure specialization. Physical plausibility, the pressure reference, and the suitability of the operating limits remain part of the data description.

A finite reading can lie outside the operating interval. Such a reading is stored and flagged rather than discarded. Invalid representation and an out-of-limit measurement are different outcomes.

3.5. The parent validates before storing common fields

Lecture-slide alignment: Week 9 source PDF, p. 60.

The slide prints only the final assignments of the initializer and describes the checks that precede them. The following complete initializer is recovered from the previously supplied Week 9 annotated companion.

```python
# Methods inside Sensor; complete class: sensor_hierarchy.py.
class Sensor:
    # One class default; individual readings belong to each instance.
    decimal_places = 1

    def __init__(self, name, unit, value, lower_limit, upper_limit):
        # Validate into local variables before storing any numeric state.
        if not isinstance(name, str) or not isinstance(unit, str):
            raise TypeError("Sensor name and unit must be strings")
        if not name.strip() or not unit.strip():
            raise ValueError("Sensor name and unit must not be empty")
        reading = checked_number(value, "Reading")
        lower = checked_number(lower_limit, "Lower limit")
        upper = checked_number(upper_limit, "Upper limit")
        if lower > upper:
            raise ValueError("Lower limit must not exceed upper limit")
        # The initializer does not call an overridable update method.
        # Store already-validated state on this same instance.
        self._name = name.strip()
        self._unit = unit.strip()
        self._value = reading
        self._lower_limit = lower
        self._upper_limit = upper
```

Code 3.2: Complete parent initialization recovered from the companion

There are three stages. First, the label and unit must be nonempty strings after surrounding whitespace is ignored. Second, the reading and both limits are converted to checked finite numbers in local variables. Third, the ordering test ensures the lower limit does not exceed the upper limit. Only then are the stored fields assigned.

This initializer does not ask `is_safe()` whether the reading is in range. Initial out-of-limit values are permitted finite data. It also avoids calling an overridable update method during base initialization. All assignments occur on the current `self`, including when that receiver is a child instance.

3.6. Getters expose meaning without handing out mutable structure

Lecture-slide alignment: Week 9 source PDF, pp. 59–60.

```python
# Methods inside Sensor; complete class: sensor_hierarchy.py.
def get_name(self):
    """Return the sensor label."""
    # Return the label, not a reference to a mutable collection.
    return self._name

def get_unit(self):
    """Return the documented unit label; this performs no conversion."""
    # Return the documented unit without converting the reading.
    return self._unit

def get_value(self):
    """Return the current numeric reading without changing state."""
    # Query the current reading on this particular receiver.
    return self._value

def get_limits(self):
    """Return the inclusive operating limits as an immutable pair."""
    # Pack the two numeric limits into an immutable tuple.
    return self._lower_limit, self._upper_limit
```

Code 3.3: Getter methods inside the complete Sensor class

These method excerpts belong inside `Sensor`; they are shown separately to make each responsibility readable. Strings and numeric values are returned directly. `get_limits()` packages the two immutable numeric boundaries in a tuple. Calling these getters does not change the reading, and the returned numeric pair cannot be used to mutate the two stored numeric fields by changing a tuple element.

A unit getter returns documentation, not a physical-unit system. The caller must still supply values that match the documented unit. The helper's finite-number checks cannot discover that a value supplied in Pa was mistakenly labelled kPa.

3.7. One update rule can serve the whole family

Lecture-slide alignment: Week 9 source PDF, p. 61.

```python
# Methods inside Sensor; complete class: sensor_hierarchy.py.
def set_value(self, proposed_value):
    """Store a finite reading, even when outside operating limits."""
    new_value = checked_number(proposed_value, "Reading")
    self._value = new_value  # Commit only after validation succeeds.

def update(self, proposed_value):
    """Keep the familiar Week 8 update interface; return None."""
    self.set_value(proposed_value)
```

Code 3.4: The explicit setter and a compatibility wrapper

The `update` method preserves the familiar Week 8 name by delegating to `set_value`. Both supported routes validate the new number before storing it. A successful operation returns `None`; the object itself is updated. If validation raises, the assignment in `set_value` is skipped.

The setter does not reject 401 merely because a particular sensor's upper operating limit is

400. It preserves a valid finite measurement so a later query can signal the exceedance. Silently keeping an older normal reading instead would conceal the event.

3.8. Queries and reports still use the actual receiver

Lecture-slide alignment: Week 9 source PDF, p. 62 and the reporting examples on pp. 67–69.

```
# Methods inside Sensor; complete class: sensor_hierarchy.py.
def is_safe(self):
    """Return whether the current reading meets both inclusive limits."""
    # Both boundaries are included in this teaching rule.
    return self._lower_limit <= self.get_value() <= self._upper_limit

def status(self):
    """Return an interpretation of the stated rule, not physical safety."""
    if self.is_safe():
        return "WITHIN LIMITS"
    return "OUTSIDE LIMITS"

def report(self):
    """Return one report string without mutating any reading."""
    digits = self.decimal_places
    return (f"{self.get_name()}: {self.get_value():.{digits}f} "
            f"{self.get_unit()} | {self.status()}")
```

Code 3.5: Inherited queries and the complete base reporting method

The inclusive decision is

$$\text{lower limit} \leq \text{current reading} \leq \text{upper limit}.$$

`status()` translates the Boolean into text. `report()` composes that text with the current label, value, unit, and selected number of decimal places. All are obtained from the same receiver.

When an inherited method runs on a child, `self` is still the child instance. An expression such as `self.get_value()` therefore uses that object's applicable interface. There is no separate parent object whose reading must be synchronized with the child.

3.9. Specialize construction using super()

Lecture-slide alignment: Week 9 source PDF, pp. 63–64.

```python
# Definition excerpt from sensor_hierarchy.py; imports are supplied there.
class TemperatureSensor(Sensor):
    """Specialize the unit and add Celsius-to-Kelvin conversion."""
    def __init__(self, name, value, lower_limit, upper_limit):
        # Initialize inherited fields on this SAME new child instance.
        super().__init__(name, "deg C", value, lower_limit, upper_limit)

    def to_kelvin(self):
        """Return the converted value; stored Celsius reading is unchanged."""
        # Convert a value for the caller; retain Celsius storage.
        return self.get_value() + 273.15
```

Code 3.6: Fix the temperature unit and add a conversion

The child initializer has four caller parameters: name, value, lower limit, and upper limit. Its call to `super().__init__` supplies the fixed `"deg C"` unit to the parent. In this single-inheritance example, `super()` provides access to the parent implementation with the same receiver. It does not construct a second `Sensor`.

Stage	Event
Evaluate the class call	Obtain the label, Celsius value, and supplied limits.
Create a child instance	The new object has type `TemperatureSensor`.
Enter the child initializer	`self` refers to that object.
Call `super().__init__`	Parent code checks and stores fields on that same object.
Finish initialization	The caller receives the initialized child instance.

```python
# ENG 101 Week 9 | Specialized construction uses one receiver
# Basis: worked trace of the slide construction; source PDF page(s): 63-64.
from sensor_hierarchy import TemperatureSensor
t = TemperatureSensor("Tank", 82.5, 0.0, 90.0)  # Unit fixed by child.
print(t.get_name(), t.get_unit(), t.get_value())
print(t.get_limits())  # Getter returns a tuple of inclusive boundaries.
print(f"Kelvin: {t.to_kelvin():.2f} K")  # No change to Celsius storage.
```

Code 3.7: Worked observation of the constructed child

```
Tank deg C 82.5
(0.0, 90.0)
Kelvin: 355.65 K
```

The temperature conversion is $T_{\mathrm{K}} = T_{\circ\mathrm{C}} + 273.15$. It returns a new numeric result; it does not overwrite Celsius storage. The hierarchy's numeric checks alone do not reject all physically impossible temperatures. The slides explicitly leave full physical admissibility outside this generic contract.

3.10. Test a conversion and the state that remains

Lecture-slide alignment: Week 9 source PDF, p. 65.

```python
# ENG 101 Week 9 | Test conversion and preserved storage
# Basis: slide example; source PDF page(s): 65.
from sensor_hierarchy import TemperatureSensor, PressureSensor
t = TemperatureSensor("Ice point", 0.0, -10.0, 90.0)
p = PressureSensor("Test pressure", 2.0, 0.0, 4.0)
assert abs(t.to_kelvin() - 273.15) <= 1e-12  # Kelvin tolerance.
assert p.to_pa() == 2000.0  # Independent conversion: 2 times 1000.
assert t.get_value() == 0.0  # Celsius storage remains unchanged.
assert p.get_value() == 2.0  # kPa storage remains unchanged.
print("Conversion and state tests passed")
```

Code 3.8: Independent unit-conversion checks and no-mutation checks

```
Conversion and state tests passed
```

The checks use an easily verified point, $0\,^{\circ}\mathrm{C} \mapsto 273.15$ K, and an easily verified scale conversion, 2 kPa $\mapsto$ 2000 Pa. The Kelvin check uses a small arithmetic tolerance. Separate assertions ensure the stored values remain 0.0 in Celsius and 2.0 in kPa.

A function can return the correct converted number and still violate the interface by changing the stored unit convention. That is why returned-value and state-preservation checks are different pieces of evidence.

3.11. A pressure specialization and a compatible override

Lecture-slide alignment: Week 9 source PDF, pp. 66–67.

The pressure child fixes the public unit to kPa, provides a conversion to Pa, chooses its own default display precision, and extends the parent's report. The following full definition is recovered from the companion; the slides print its conversion and reporting excerpts.

```python
# Definition excerpt from sensor_hierarchy.py; imports are supplied there.
class PressureSensor(Sensor):
    """Specialize the unit and add kPa-to-Pa conversion and report text.

    The pressure reference is part of the supplied data description. This
    class preserves the generic finite-number contract (signed values allowed).
    """
    decimal_places = 0  # Child-level default; Sensor's default stays 1.

    def __init__(self, name, value, lower_limit, upper_limit):
        super().__init__(name, "kPa", value, lower_limit, upper_limit)

    def to_pa(self):
        """Return the value in Pa; extreme overflow is rejected explicitly."""
        result = self.get_value() * 1000.0
        return checked_number(result, "Converted pressure in Pa")

    def report(self):
        """Extend, rather than discard, the inherited report contract."""
        base_text = super().report()  # This does not call this override again.
        try:
            converted_text = f"{self.to_pa():.0f} Pa"
        except ValueError:
            # Keep reporting the valid kPa reading if its conversion overflows.
            converted_text = "Pa conversion unavailable"
        return f"{base_text} | {converted_text}"
```

Code 3.9: A complete pressure specialization with checked conversion and report extension

The `decimal_places = 0` assignment belongs to the child class. It does not overwrite the parent's default. The conversion multiplies the stored kPa value by 1000 and checks that the derived number is representable as a finite float. Changing units does not change whether the original data describe absolute or gauge pressure.

In the reporting override, `super().report()` selects inherited reporting behavior rather than recursively calling the same override. Its result becomes `base_text`. A successful conversion contributes an additional Pa field. A conversion `ValueError` contributes an explicit unavailable-value message instead. In either route the method returns a string and retains the original kPa reading.

Engineering check

A failed derived conversion is not a reason to replace a valid original observation with zero. Retaining the kPa report while explicitly marking the Pa value unavailable preserves information and the reporting contract.

3.12. Trace the method call and the return path

Lecture-slide alignment: Week 9 source PDF, p. 68.

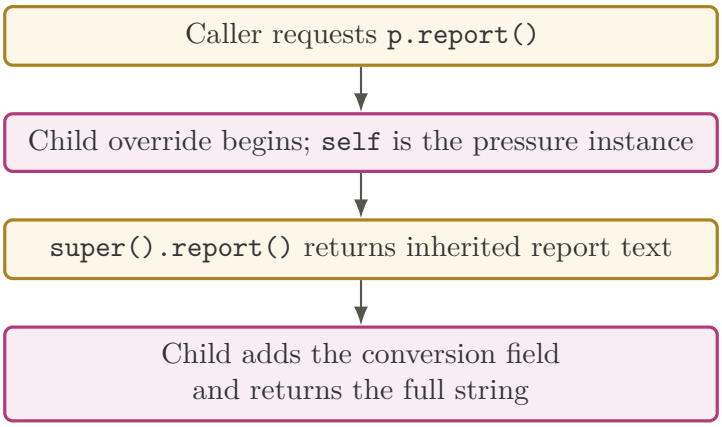


Figure 3.2.: The dispatch-and-return sequence from source page 68, expanded without inventing a separate parent object.

For a pressure sensor with value 315 and limits 100–400, the base text uses the pressure child's zero-decimal setting and the parent's inclusive-limit logic. The override then appends `315000 Pa`. The parent report is reused, not reproduced line by line in the child.

3.13. Polymorphism: one loop, applicable behavior

Lecture-slide alignment: Week 9 source PDF, p. 69.

```python
# ENG 101 Week 9 | One interface selects applicable behavior
# Basis: slide example; source PDF page(s): 69.
from sensor_hierarchy import Sensor, TemperatureSensor, PressureSensor
sensors = [
    TemperatureSensor("Tank", 82.5, 0.0, 90.0),
    PressureSensor("Pipe", 315.0, 100.0, 400.0),
    Sensor("Battery", "V", 10.8, 11.0, 12.8),
]
for sensor in sensors:
    print(sensor.report())  # No separate exact-type decision tree needed.
```

Code 3.10: A list of related objects uses one reporting interface

```
Tank: 82.5 deg C | WITHIN LIMITS
Pipe: 315 kPa | WITHIN LIMITS | 315000 Pa
Battery: 10.8 V | OUTSIDE LIMITS
```

Polymorphism here means that the same requested operation, `report()`, selects behavior appropriate to each object's class. The loop does not need repeated branches checking whether a particular object is temperature, pressure, or generic. This works because all three provide compatible returned text.

The battery's 10.8 V is a finite stored value but lies below its supplied 11.0 V lower limit. The output therefore reports it rather than hiding it. The pressure report adds an extra conversion field while maintaining the parent's original label, unit, status, and returned-string meaning.

3.14. Exact type and inheritance answer different questions

Lecture-slide alignment: Week 9 source PDF, p. 70.

```python
# ENG 101 Week 9 | Exact type differs from membership
# Basis: slide example; source PDF page(s): 70.
from sensor_hierarchy import Sensor, TemperatureSensor, PressureSensor
t = TemperatureSensor("Tank", 20.0, 0.0, 90.0)
print(type(t) is TemperatureSensor)  # Exact runtime class.
print(type(t) is Sensor)  # Parent is not the exact class of t.
print(isinstance(t, Sensor))  # Subclass instances satisfy this check.
print(issubclass(TemperatureSensor, Sensor))  # Compare classes.
print(isinstance(t, PressureSensor))  # Siblings are not interchangeable types.
```

Code 3.11: Exact runtime type, instance membership, and subclass relationships

```
True
False
True
True
False
```

`type(t) is TemperatureSensor` asks whether that is the exact class object. `isinstance(t, Sensor)` permits instances of a subclass, so it can be true at the same time as `type(t) is Sensor` is false. `issubclass` compares class relationships rather than individual readings. Two children of a common parent are siblings; a temperature instance does not therefore become a pressure instance.

These are inspection tools, not a requirement to write exact-type branches in every processing loop. A compatible common interface often makes such branching unnecessary.

3.15. Defaults can be inherited or shadowed again

Lecture-slide alignment: Week 9 source PDF, p. 71.

```python
# ENG 101 Week 9 | Trace parent, child, and instance settings
# Basis: slide example; source PDF page(s): 71.
from sensor_hierarchy import Sensor, TemperatureSensor, PressureSensor
Sensor.decimal_places = 1
PressureSensor.decimal_places = 0  # Child class default shadows parent's.
t = TemperatureSensor("Tank", 20.0, 0.0, 90.0)
p = PressureSensor("Pipe", 315.0, 100.0, 400.0)
Sensor.decimal_places = 2
print(t.decimal_places, p.decimal_places, Sensor.decimal_places)
p.decimal_places = 3  # Override only this pressure instance.
print(p.decimal_places, PressureSensor.decimal_places)
```

Code 3.12: Parent default, child default, and one instance override

```
2 0 2
3 0
```

Temperature has no own `decimal_places` class setting here, so it follows the parent's new value

of two. Pressure has a child-level zero, which shadows the parent. Finally, the assignment of three to `p.decimal_places` overrides the child setting for only that one instance. The class `PressureSensor` still stores zero.

Location	Final ordinary setting used in the example
Parent `Sensor`	2
Child `PressureSensor`	0
Temperature instance `t`	Reads the inherited parent value 2
Pressure instance `p`	Reads its own override 3

The parent reporting method's use of `self.decimal_places` is what allows these policies to remain visible in inherited behavior.

3.16. Reusing a name is not preserving a contract

Lecture-slide alignment: Week 9 source PDF, p. 72.

A child should remain usable through the promised parent operations. Compatible input meaning, returned types, units, side effects, and failure behavior all matter. An override that prints text but returns `None` violates the parent's report interface even though its name is still `report`.

Worked completion / extension

The slide asks students to inspect "example 26" but does not print that file. The following is a labelled reconstruction of the described defect, not a recovered verbatim listing. The parent `Sensor` itself is the supplied companion implementation.

```python
# ENG 101 Week 9 | A reporting override loses the returned string
# Basis: labelled reconstruction of the named but unprinted example 26; source PDF page(s): 72.
from sensor_hierarchy import Sensor
class PrintedReportSensor(Sensor):
    def report(self):
        print(super().report())  # BUG: prints the inherited text, returns None.
s = PrintedReportSensor("Tank", "deg C", 20.0, 0.0, 90.0)
result = s.report()  # Output appears, but the caller receives no report string.
print(result)
```

Code 3.13: Reconstructed defect: the override prints but loses the return value

```
Tank: 20.0 deg C | WITHIN LIMITS
None
```

The minimal repair is to return the inherited string, or return an intentionally extended string. Merely seeing a message in the terminal does not satisfy a caller that needs to compose, store, or test the report.

3.17. Test inherited behavior, not just the added method

Lecture-slide alignment: Week 9 source PDF, p. 73.

```python
# ENG 101 Week 9 | Inherited contracts require tests
# Basis: slide example; source PDF page(s): 73.
from sensor_hierarchy import PressureSensor
p = PressureSensor("Pipe", 315.0, 100.0, 400.0)
p.update(400.0)  # Inclusive upper boundary.
assert p.is_safe()
p.update(401.0)  # Valid finite reading: retain it and flag the limit breach.
assert not p.is_safe()
before = p.get_value()
try:
    p.update("bad")
except TypeError:
    assert p.get_value() == before  # Failure must not erase the last value.
else:
    raise AssertionError("Expected a TypeError")
print("Inherited behavior tests passed")
```

Code 3.14: Boundary, valid warning, and rejected update in a child instance

```
Inherited behavior tests passed
```

The 400.0 update reaches the included upper boundary and remains within limits. The 401.0 update is accepted finite data and produces an out-of-limit status. A text update is rejected with `TypeError`, and the object must still store 401.0 afterward. The class is not tested adequately by checking only `to_pa()`.

Worked completion / extension

The next check exercises the conversion-overflow branch explicitly described on source pages 66–67. The magnitude is a numerical stress test, not a plausible physical operating example.

```python
# ENG 101 Week 9 | Keep valid data when a derived conversion overflows
# Basis: worked numerical edge test of the slide report contract; source PDF
    page(s): 66-67.
from sensor_hierarchy import PressureSensor
p = PressureSensor("Range check", 1e308, 0.0, 1e308)
before = p.get_value()  # Extreme numerical test, not a physical scenario.
try:
    p.to_pa()  # Multiplication by 1000 exceeds floating-point range.
except ValueError:
    print("Conversion rejected")
report = p.report()  # Report handles the missing derived value explicitly.
print("Pa conversion unavailable" in report)
print(p.get_value() == before)  # The original kPa value remains valid data.
```

Code 3.15: Retain the original kPa observation when a derived conversion overflows

```
Conversion rejected
True
True
```

3.18. Debugging: the parent initializer never ran

Lecture-slide alignment: Week 9 source PDF, p. 74.

```python
# ENG 101 Week 9 | A replacement initializer omits the parent
# Basis: slide example; source PDF page(s): 74.
from sensor_hierarchy import Sensor
class TemperatureSensor(Sensor):
    def __init__(self, name, value):
        self._unit = "deg C"  # BUG: never initializes _value and other fields.
t = TemperatureSensor("Tank", 20.0)
print(t.get_value())  # Expected AttributeError: _value is absent.
```

Code 3.16: Intentional exception: a child replaces initialization without establishing inherited state

```
AttributeError (intentional error)
```

The child's own initializer runs, storing only `_unit`. Defining that initializer replaces the automatically selected inherited initializer; Python does not then call the parent initializer as an extra implicit step. Consequently `get_value()` cannot find `_value`.

Repair by calling `super().__init__` with the name, fixed unit, value, and appropriate supplied limits, as in the complete temperature specialization. Adding a random `_value` assignment alone would not establish all the parent's promised fields and checks. The smallest responsible repair restores the intended initialization path, not merely the first attribute mentioned by the exception.

3.19. Is-a and has-a are different relationships

Lecture-slide alignment: Week 9 source PDF, p. 75.

A pressure sensor *is a* sensor: inheritance can represent this specialization. An experiment *has* sensors: it should not become a sensor merely because it contains them. Similarly, a bar has a material. This containment relationship is **composition**.

```python
# Definition excerpt from sensor_hierarchy.py; imports are supplied there.
class Experiment:
    """An experiment HAS sensors; it is not itself a Sensor."""
    def __init__(self, name, sensors):
        self.name = name
        # Copy the outer list, not the sensor objects it refers to.
        self._sensors = list(sensors)

    def reports(self):
        """Return reports through the common interface."""
        return [sensor.report() for sensor in self._sensors]
```

Code 3.17: Recovered companion: an experiment contains references to sensors

The initializer copies the outer input list. It does not copy the sensor objects. The reports method uses each referenced sensor's common interface. Its list comprehension has the same meaning as creating an empty results list and appending one returned report for each sensor.

```python
# ENG 101 Week 9 | Copy the list, retain the sensor references
# Basis: worked completion using the supplied companion Experiment class; source PDF page(s): 75.
from sensor_hierarchy import Experiment, TemperatureSensor
t = TemperatureSensor("Tank", 20.0, 0.0, 90.0)
original = [t]
experiment = Experiment("Run 1", original)  # Copy the outer list only.
original.clear()  # Does not remove t from the experiment's copied list.
t.update(95.0)  # The experiment still refers to this same sensor object.
print(experiment.reports()[0])
```

Code 3.18: Worked example: the list is independent, the sensor remains shared

```
Tank: 95.0 deg C | OUTSIDE LIMITS
```

Clearing the caller's original list does not remove the sensor from the experiment's copied list. Updating the shared temperature object does change the reading seen through the experiment. This distinction combines composition with the Week 5 shallow-copy model.

3.20. Worked activity: add a VoltageSensor

Lecture-slide alignment: Week 9 source PDF, p. 76.

The activity requires a child of `Sensor`, unit `"V"` supplied through the parent initializer, a conversion to millivolts that does not overwrite stored volts, and tests of a known conversion and inherited limit.

```python
# Worked activity solution; imports: activity_solutions.py.
class VoltageSensor(Sensor):
    """Activity solution: inherit sensor behavior and convert V to mV."""
    def __init__(self, name, value, lower_limit, upper_limit):
        # Reuse parent validation while fixing the documented unit.
        super().__init__(name, "V", value, lower_limit, upper_limit)
    def to_millivolts(self):
        # Activity assumption: this converted result is representable.
        return self.get_value() * 1000.0
```

Code 3.19: Activity solution: specialize only the unit and the additional conversion

```python
# ENG 101 Week 9 | Add a conversion without rewriting the parent
# Basis: worked solution to the slide activity; source PDF page(s): 76.
from activity_solutions import VoltageSensor
v = VoltageSensor("Battery", 12.0, 11.0, 12.8)
assert v.to_millivolts() == 12000.0  # Independent V-to-mV answer.
assert v.get_value() == 12.0  # Conversion is a query, not an update.
v.update(12.8)
assert v.is_safe()  # Inherited inclusive boundary still applies.
print("Voltage sensor tests passed")
```

Code 3.20: Verify 12 V to 12000 mV and an inherited boundary

```
Voltage sensor tests passed
```

The solution inherits update, getters, status, and report instead of copying them. Its conversion assumes the multiplied result is representable, which is sufficient for the activity values. A more general conversion contract would need to state and check its supported numerical range; that is not silently added to this short activity answer.

3.21. Integrate evidence and state the limits

Lecture-slide alignment: Week 9 source PDF, p. 77.

For a useful review of the sensor family, record an independently known conversion, an inclusive boundary, a rejected update with unchanged state, and an independence check between instances. Explain what labels and units mean. Identify the parent's shared guarantees and each child's additional behavior.

Passing these tests supports the selected software behavior. It does not certify a sensor, establish calibration accuracy, supply an authoritative operating threshold, or validate every input. The source explicitly identifies this integration as practice rather than a new graded task.

3.22. Lecture 27 summary and exit-ticket answers

Lecture-slide alignment: Week 9 source PDF, p. 78.

Section summary

Inheritance reuses a contract and its behavior on the same instance. `super()` accesses inherited behavior; it does not create another parent object. Specialized methods should preserve returned types, units, side effects, and rejection guarantees. Composition represents contained collaborators rather than a subtype relationship.

For the exit ticket: `super().__init__(...)` initializes inherited state on the existing child instance. `isinstance(t, Sensor)` can be true because it accepts subclass instances even though the exact type is `TemperatureSensor`. A reporting override must still return a string because client code relies on that contract. An experiment that owns sensors has a *has-a* relationship with them.

4 Optional Features and Week 9 Synthesis

Lecture-slide alignment: Week 9 source PDF, pp. 79–87.

The source labels properties, the name-mangling demonstration, and string representations as optional. They are included here so that the notes cover the complete presentation without turning the optional material into extra core requirements. Explicit access methods and the small single-inheritance hierarchy remain sufficient for the principal examples.

4.1. Attribute-like access through a property

Lecture-slide alignment: Week 9 source PDF, pp. 79–80.

A **property** provides attribute-like syntax that is implemented by methods. In the source example, reading `p.value_kpa` invokes a getter; assigning `p.value_kpa = x` invokes a setter. The external syntax resembles an ordinary attribute, but the supported route executes validation.

```python
# Definition excerpt from optional_models.py; imports are supplied there.
class PressureProperty:
    """One absolute pressure in kPa, exposed through a validated property."""
    def __init__(self, value_kpa):
        self.value_kpa = value_kpa  # Assignment invokes the property setter.

    @property
    def value_kpa(self):
        """Return the current pressure in kPa."""
        # Return the stored numeric value; leave the object unchanged.
        return self._value_kpa

    @value_kpa.setter
    def value_kpa(self, proposed_kpa):
        # Use a DIFFERENT backing name; self.value_kpa would recurse here.
        new_value = checked_nonnegative(proposed_kpa, "Absolute pressure")
        self._value_kpa = new_value

    def __str__(self):
        """Supply readable text for print(instance)."""
        return f"Absolute pressure: {self.value_kpa:.1f} kPa"

    def __repr__(self):
        """Supply a diagnostic description for repr(instance)."""
        return f"PressureProperty({self.value_kpa!r})"
```

Code 4.1: Optional property class, including the representation methods explained below

The decorator `@property` establishes a getter. Its public name is `value_kpa`. The second decorator associates the setter with assignment to that same public name. You do not pass `self` at the call site or add parentheses when reading the property.

The initializer assigns the public property, so construction goes through the setter. The setter

validates a local proposed value and writes to the distinct internal name `_value_kpa`. This preserves the data invariant for supported property assignments. The complete class above is recovered from the Week 9 companion; source page 79 prints the property portion and source page 83 prints the two representation methods.

```python
# ENG 101 Week 9 | Read and assign through a property
# Basis: slide example; source PDF page(s): 80.
from optional_models import PressureProperty
p = PressureProperty(315.0)
p.value_kpa = 400.0  # Invokes the property setter, not plain assignment.
print(p.value_kpa)  # Invokes the property getter.
try:
    p.value_kpa = -1.0
except ValueError:
    print("Rejected")
print(p.value_kpa)  # Previous valid pressure remains.
```

Code 4.2: Reading, accepted assignment, and rejected assignment through the property

```
400.0
Rejected
400.0
```

Do not call the property setter from inside itself by accident

Writing `self.value_kpa = proposed_kpa` inside this setter invokes the same setter again. Without a different terminating route, calls continue until Python raises `RecursionError`. The internal backing assignment must instead use `self._value_kpa`. The separately supplied `property_recursion.py` is an intentional-error illustration of this slide warning.

A property is not an encryption or security mechanism. A caller can still deliberately access backing names. It controls the public property route. It also shows why the ordinary lookup diagram in Chapter 2 is deliberately limited: not every attribute read or assignment is an ordinary unmediated instance operation. Python's property documentation provides the narrow supplementary reference [4]; descriptor internals are not required here.

4.2. Locate the complete numeric helper

Lecture-slide alignment: Week 9 source PDF, p. 81.

Source page 81 prints `checked_number`. The complete commented version is presented earlier in Section 1.6, where it is first needed, and is supplied in `examples/numeric_checks.py`. Its key steps are type checking, float normalization, conversion-overflow handling, rejection of NaN and infinities, and a returned finite value.

The companion's `checked_nonnegative` then rejects negative results. No external numeric package is required. Validity checks remain distinct from the operating-limit decisions performed by the generic sensor hierarchy.

4.3. Name mangling changes a name, not accessibility

Lecture-slide alignment: Week 9 source PDF, p. 82.

```python
# ENG 101 Week 9 | Mangled names are not a security boundary
# Basis: slide example; source PDF page(s): 82.
class Device:
    def __init__(self):
        self.__code = "A"  # Mangled inside this class definition.
    def get_code(self):
        return self.__code  # Public method uses the internal name.
item = Device()
print(item.get_code())  # Intended interface.
print(item._Device__code)  # Demonstrate accessibility, not normal client use.
```

Code 4.3: Optional demonstration: both accesses reach the same stored code

```
A
A
```

Inside this `Device` definition, `__code` is transformed to a class-qualified internal name such as `_Device__code`. This reduces accidental collisions when subclasses use their own implementation names. It does not prevent a caller who knows the transformed name from reading it.

The public getter is the intended client interface. The explicit mangled-name access is shown only to demonstrate the limitation. Do not conclude that a double underscore makes a password, measurement, or proprietary result secret. A name such as `__init__`, with double underscores at both ends, is a Python special-method convention rather than this example's private-name pattern.

4.4. Readable text and diagnostic text

Lecture-slide alignment: Week 9 source PDF, p. 83.

The optional `PressureProperty` class above implements both `__str__` and `__repr__`. The first supplies readable text for ordinary display. The second supplies a diagnostic description when the caller requests `repr(instance)`.

```python
# ENG 101 Week 9 | Readable and diagnostic text
# Basis: slide example; source PDF page(s): 83.
from optional_models import PressureProperty
p = PressureProperty(315.0)
print(p)  # Uses __str__ to obtain readable text.
print(repr(p))  # Uses __repr__ to obtain diagnostic text.
assert p.value_kpa == 315.0  # Neither representation changes the reading.
```

Code 4.4: Compare the two representations without changing state

```
Absolute pressure: 315.0 kPa
PressureProperty(315.0)
```

Both special methods return strings. They do not print internally or update the pressure. In the diagnostic f-string, `!r` requests the representation of the embedded value. The class-like

appearance of a representation is useful for inspection, but is not a promise that every object can be reconstructed by executing its text. Nothing in this exercise requires `eval` or execution of a representation string.

4.5. Interface and inheritance quick reference

Lecture-slide alignment: Week 9 source PDF, p. 84.

Syntax	Interpretation in these examples
`obj.get_value()`	Call a public getter and use its returned result.
`obj.set_value(x)`	Request a controlled update; inspect its contract.
`ClassName.setting`	Read or explicitly assign an attribute on the named class.
`obj.setting = x`	For an ordinary attribute, can create an instance override; a property may instead invoke its setter.
`class Child(Parent):`	Define a specialized type with inherited behavior.
`super().method(...)`	Continue lookup to inherited behavior on the same receiver.
`isinstance(obj, Parent)`	Test parent-or-subclass membership.
`issubclass(Child, Parent)`	Test the relationship between classes.
`self._value`	Access a non-public backing attribute by convention.
`del obj.setting`	Remove an ordinary instance binding in the shadowing examples.

The table includes two explicitly explained conveniences from the core slides, instance deletion and `issubclass`, alongside the source reference table. The meaning of any line still depends on the class and its contract; syntax alone does not decide a good design.

4.6. Suggested reading and the source basis

Lecture-slide alignment: Week 9 source PDF, p. 85.

The presentation's own reading list names the ENG 101 Course Specification, Session 9; Wang's supplied *Programming for Engineers with Python*, Sections 15.13 and 15.18–15.24, together with Appendix H.8–H.14 and H.22–H.27; and Guttag's second edition, Sections 8.2–8.3. Its technical checks cite the Python class tutorial and built-in `property` and `super` documentation.

These notes are based on the attached slide deck and its earlier supplied annotated companion, rather than claiming every statement was independently extracted from those larger works. The books remain suggested reading as identified by the deck. The final source appendix distinguishes the direct basis, recovered definitions, activity solutions, and supplementary technical qualifications.

4.7. Synthesis: one set of promises across three ideas

Lecture-slide alignment: Week 9 source PDF, pp. 86–87.

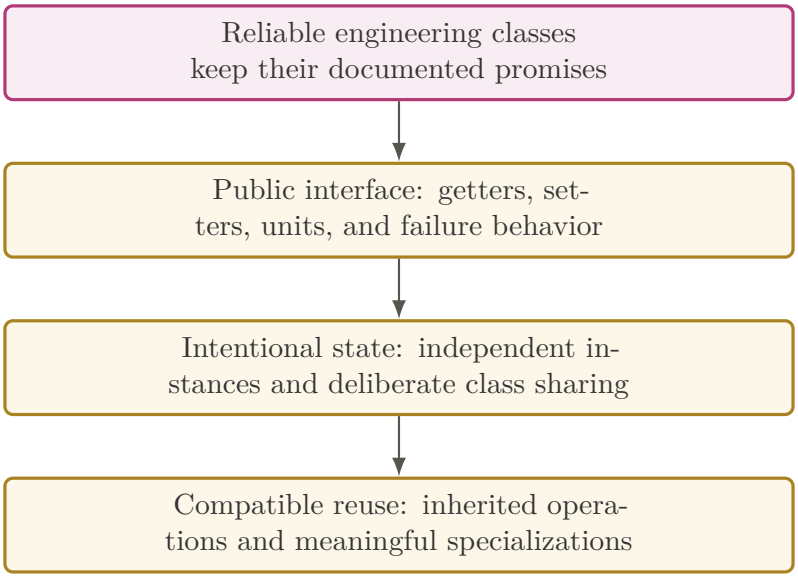


Figure 4.1.: The source page 86 synthesis expanded into a reading checklist.

For each part, identify one defect and one test. A setter that assigns before validation needs a failure-state test. A class-level history used unintentionally needs an independence test. A child that prints rather than returns needs a returned-type test. A changed storage representation needs public-unit and numerical-range checks.

Closing message from Week 9

A reusable class keeps its promises. Make state changes intentional, shared data visible, and specializations compatible. The next specified unit introduces orders of growth and Big-O notation; correctness and clear contracts remain prerequisites for meaningful efficiency comparisons.

4.8. Readiness checklist

You are ready to move on when you can explain an interface without exposing its backing names, trace a successful and failed setter call, distinguish an instance override from a class assignment, draw the references behind a shared list, state a counter's meaning, follow `super()` on the same receiver, and test a child through inherited operations. Be able to explain *why* an answer follows from the state and lookup path, not merely recall a printed value.

5 Cumulative Practice and Worked Solutions

The following 12 questions are additional self-study practice based on the Week 9 concepts. They are not new assessment requirements or a prediction of examination questions. Attempt the questions before consulting the solutions. Use the supplied model definitions where a class is imported; keep intentionally weak miniature classes separate from the validated reference models.

5.1. Practice questions

Q1. Track the setter's result and state

A `PressureReading` starts at 315 kPa. The statement `result = p.set_value_kpa(400)` executes successfully. What is `result`? What does the next getter return? Explain why assigning that result back to `p` would be a different action.

Q2. Write a rejection test that can actually fail

Design a test starting from 100 kPa and proposing −1 kPa. It must fail if the proposal is wrongly accepted, and it must fail if the old state is overwritten before rejection. State the exception expected for `True` under the same class's contract.

Q3. Is a returned list independent?

Start from `ReadingSeries([20.0, 21.0])`. Save a result from its copying getter, then add 22.0 to the series using `add_reading`. What appears in the saved list and in a new getter result? Would the same shallow-copy argument protect arbitrary nested mutable objects?

Q4. Trace three ordinary settings

A class has `digits = 2`; instances `a` and `b` are created. Execute `a.digits = 4`, then `ClassName.digits = 1`, then `del a.digits`. Give the values read by `a`, `b`, and the class after each step.

Q5. Separate mutation from rebinding

Use the source `Store` example from Section 2.11. After its final printed lines, execute `b.readings.append(25.0)`. Which of the class list, `a.readings`, and `b.readings` changes? Give all three final lists.

Q6. Interpret a count conservatively

With `CountedReading.total_created` set to zero in a fresh demonstration, create valid instances from 10 and 20, reject a text proposal, and then execute `del a`. What is the counter? Why is it not a live-object counter or a permanent identifier?

Q7. Distinguish a valid warning from invalid input

A generic `PressureSensor` contains 315 kPa with limits 100–400 kPa. Update it to 401 kPa, then attempt to update with `"bad"`. State the value and status after each attempt. Compare this object's domain with the standalone absolute-pressure class.

Q8. Construct a child without inventing another object

A temperature sensor named Tank is created with value 20.0 and limits 0.0–90.0 in Celsius. Explain each stage of that class call. What does `super().__init__` do? What is the Kelvin result? Which stored value remains afterward? Which `isinstance` and exact-type tests should be true?

Q9. Repair a reporting override

A child method executes `print(super().report())` and contains no explicit return. A caller appends `s.report()` to a list of reports. What is appended? Show the smallest repair that retains the parent report's no-mutation and returned-string promises.

Q10. Reason about composition

An experiment is constructed from a one-element list containing a temperature sensor. The original list is cleared and the sensor is updated. Why can the experiment still report that sensor, and why does it see the updated value? State separately what was copied and what remained shared.

Q11. Optional: why does the property recurse?

Inside a `value_kpa` property setter, a programmer writes `self.value_kpa = proposed_kpa`. Explain the repeating call route, the expected eventual exception, and the correct backing-name pattern. Does changing the backing field to a double-leading-underscore name create a security boundary?

Q12. Complete a compact engineering object test plan

Using the worked `VoltageSensor`, propose tests for a known conversion, a lower boundary, an upper boundary, one finite out-of-limit value, one invalid type, and preservation of the previous state. Explain why these tests do not establish calibration accuracy or certified physical safety.

5.2. Worked solutions

Solution 1: a changed object and a None result

The successful setter returns `None`, while the getter returns 400.0 kPa. The statement stores the return value in `result`; it does not replace the existing object. Assigning it to `p` would rebind the caller's name to `None`. The earlier reference no longer identifies the updated pressure object through that name. This is the same distinction as a list operation that mutates a list but does not return a replacement list.

Solution 2: assert both the exception route and the remaining state

```
# Use the supplied class; it validates before changing stored pressure.
from access_models import PressureReading
p = PressureReading(100.0)
before = p.get_value_kpa()  # Independent record of the prior state.
try:
    p.set_value_kpa(-1.0)
except ValueError:
    assert p.get_value_kpa() == before  # Detect mutation before failure.
else:
    raise AssertionError("Expected rejection of negative pressure")
```

Code 5.1: Practice solution: a rejected proposal must leave the original state

A `True` proposal is rejected with `TypeError`, because the measurement helper explicitly excludes Boolean values. A test that only prints an exception message would not verify state preservation, and a test that lacks the `else` branch could quietly pass when the invalid call unexpectedly succeeds.

Solution 3: the old snapshot remains unchanged

The saved list remains `[20.0, 21.0]`. A new getter call returns `[20.0, 21.0, 22.0]`. The copying getter creates a fresh outer list, while `add_reading` changes the object's internal list. For the flat immutable numbers here, the separation is sufficient. With nested mutable elements, those elements can still be shared by a shallow copy; a broader snapshot guarantee would require a different explicit design.

Solution 4: record class and instances separately

After	a reads	b reads	Class
Creation	2	2	2
`a.digits = 4`	4	2	2
`ClassName.digits = 1`	4	1	1
`del a.digits`	1	1	1

The deleted binding was only the instance override. The class remains defined and stores its most recent value.

Solution 5: only b's new list changes

The class list and `a.readings` remain `[10.0, 30.0]`. The value of `b.readings` becomes `[20.0, 25.0]`. The earlier rebinding on `b` already created a separate list and ordinary instance attribute. Later appending through `b` therefore mutates that list, not the class-level list. Always resolve the attribute before deciding which object an operation modifies.

Solution 6: the count stays at two

Two successful initializations increment the counter; the invalid proposal does not. Deleting the name `a` does not call any code that decrements `total_created`. The variable records successful initializations in this run. It does not track every object's lifetime, and its value can be reset when a process restarts, so it is not a permanent or distributed identity mechanism.

Solution 7: retain the valid observation and reject the invalid proposal

After 401.0 the sensor stores 401.0, `is_safe()` returns `False`, and `status()` returns `"OUTSIDE LIMITS"`. The text proposal raises `TypeError`; afterward the same value and status remain. The generic hierarchy accepts finite signed readings, whereas `PressureReading` additionally requires nonnegative absolute pressure. Neither should be silently treated as the other's input contract.

Solution 8: parent code initializes the child receiver

The arguments are evaluated, a child instance is created, and its initializer receives that instance as `self`. The call to `super().__init__` runs the parent validation and assignments on the same object. The child finishes and the caller receives it. The conversion gives 293.15 K while stored Celsius remains 20.0. Its exact type is `TemperatureSensor`, and `isinstance(t, Sensor)` is true. Its exact type is not `Sensor`.

Solution 9: return the promised report

The faulty override prints a message but implicitly returns `None`, so `None` is appended. A compatible repair returns the inherited text:

```
# The supplied parent already returns a report string without mutation.
from sensor_hierarchy import Sensor
class ReportSensor(Sensor):
    def report(self):
        # Forward the parent's string; do not replace it with printed output.
        return super().report()
```

Code 5.2: Practice solution: a compatible report override

For real specialization, the child could construct and return extended text from this result. If nothing changes, the method can simply be inherited rather than overridden.

Solution 10: independent container, shared contained objects

The `Experiment` initializer copies the outer list using `list(sensors)`. The original list can be cleared without clearing that copy. Both the caller and the experiment still refer to the same sensor object, so an update to that object is visible in its report. This is not a deep snapshot of

devices. Composition here means an experiment has sensor collaborators, not that it is a subtype of `Sensor`.

Solution 11: assignment invokes the same property again

Assigning the public property inside its own setter re-enters that setter with the same receiver and proposal. Without a terminating alternative, the call chain reaches the recursion limit and raises `RecursionError`. Validate to a local variable and then assign a distinct backing field such as `self._value_kpa`. A double-leading underscore may mangle the backing name; it still does not make data inaccessible or encrypted.

Solution 12: known answer, boundaries, warning, and rejection

Test	Input / operation	Expected observation
Known conversion	12.0 V	12000.0 mV; stored value remains 12.0 V.
Lower boundary	Update to 11.0 V	Accepted; within limits.
Upper boundary	Update to 12.8 V	Accepted; within limits.
Valid warning	Update to 10.8 V	Accepted; outside limits.
Invalid type	Update with `"bad"`	`TypeError`; old value preserved.

```python
# Complete the Week 9 activity with tests, not copied parent methods.
from activity_solutions import VoltageSensor
v = VoltageSensor("Battery", 12.0, 11.0, 12.8)
assert v.to_millivolts() == 12000.0
assert v.get_value() == 12.0  # A conversion must not replace stored volts.
for boundary in [11.0, 12.8]:
    v.update(boundary)
    assert v.is_safe()  # Both boundaries are included.
v.update(10.8)
assert not v.is_safe()  # A valid out-of-limit observation is retained.
before = v.get_value()
try:
    v.update("bad")
except TypeError:
    assert v.get_value() == before  # Rejection must preserve old state.
else:
    raise AssertionError("Expected TypeError")
```

Code 5.3: Practice solution: test the voltage child through inherited operations

These tests check the declared computation and state behavior for selected cases. Calibration, uncertainty, device specifications, and the physical suitability of 11.0–12.8 V are separate evidence. The numbers remain illustrative teaching limits.

6 Week 9 Glossary

These definitions use the terminology of the lecture deck. Read them together with the model-specific contracts rather than as claims about every possible Python class.

Attribute
A named item accessed through dot notation on an object or class. It may be a stored value or provide behavior such as a method or property.

Backing attribute
The internal name used to store a value behind an interface; for example, `_value_kpa` behind a getter and setter.

Class attribute / class variable
An attribute associated with the class object, such as a default report precision or a deliberately shared initialization count.

Client code
Code that uses an object's documented interface. Its ordinary work should not depend on internal storage names.

Composition
A has-a relationship in which one object contains or refers to collaborators, such as an experiment holding sensor references.

Contract
A stated agreement about valid inputs, units, results, side effects, and failure behavior.

Getter
An operation that returns a promised value. Its name alone does not guarantee that it is non-mutating or that a returned collection is copied.

Information hiding
Design in which client code depends on documented operations rather than the exact stored representation.

Inheritance
A relationship in which a specialized class obtains behavior from a more general class. Compatible reuse preserves interface meaning.

Initialization
Establishing the expected state of a newly created instance. Parent code can initialize inherited fields on the same child receiver.

Instance attribute
An attribute associated with a particular instance, such as its current reading or independent history.

Invariant
A condition promised for each valid stored state before and after supported operations.

Is-a relationship
A specialization relationship suitable for inheritance when behavior is compatible; a temperature sensor is a sensor.

Has-a relationship
A containment or collaboration relationship; an experiment has sensors rather than being

one of them.

Method override
A method in a child class that supplies an implementation under an inherited method name. Matching the name does not by itself preserve the contract.

Mutation
Changing an existing object, such as appending to a shared list. It differs from binding a name or attribute to another object.

Name mangling
A transformation of suitable double-leading-underscore names within class definitions to reduce accidental collisions. It is not secrecy or encryption.

Non-public convention
A naming signal, often a single leading underscore, that ordinary client code should not rely on an implementation detail.

Ordinary attribute lookup
The limited instance-then-class model used for the plain attributes in Lecture 26; not the complete algorithm for properties or other advanced mechanisms.

Parent / base / superclass
The general class from which a specialized child inherits behavior.

Child / derived / subclass
A specialized class that inherits from a parent and may add or override behavior.

Polymorphism
Use of a common requested operation with the applicable implementation selected for each object; the reporting loop is the central example.

Property
An attribute-like interface implemented through methods for reading and optionally assignment. It can execute validation on public assignment.

Public interface
The operations and behaviors intended for client use, with explicit quantity, units, returned results, side effects, and errors.

Receiver
The particular instance on which an ordinary method operates; it is conventionally named `self` inside the method.

Rebinding
Associating an existing name or attribute with a different object. An ordinary instance assignment can shadow a class setting.

Regression test
A retained check that a repaired defect stays fixed. It can verify a returned result, failure category, or preserved state.

Representation
The internal organization or unit convention used for stored data; it may differ from the public unit.

Setter
A controlled state-change request. In these examples successful setters mutate the existing instance and return `None`.

Shadowing / instance override
: An ordinary instance attribute with the same name as a class attribute, which is selected by the local lookup model instead of the class default.

Shallow copy
: A new outer collection that still refers to the existing elements. It is enough for the flat immutable numeric histories used here, not for arbitrary nested mutable snapshots.

Snapshot
: A returned collection representing the values at a particular time, rather than a live connection to subsequent stored changes.

State preservation on failure
: A guarantee that a rejected operation leaves the previously valid state unchanged; it requires testing separately from the exception itself.

super()
: A proxy for continuing method lookup to inherited behavior. In the single-inheritance examples it selects parent behavior on the same receiver, not a new parent object.

Successful-initialization count
: The explicit meaning of `CountedReading.total_created` in the standalone example. It does not count live objects or rejected attempts.

A Sources, Adaptations, and Slide Coverage

A.1. Direct basis and reconstruction boundaries

The attached Week 9 PDF is the primary source for the subject matter, order, examples, terminology, activities, and optional labels. Complete code that the PDF imports or excerpts is recovered from the earlier supplied Week 9 annotated companion in this conversation. The reading design reuses the established Week 8 reading-note formatting and branding assets, not its teaching content.

Material	How it is used
Week 9 slides	Every source PDF page is mapped below. Narrative expansions explain the displayed contracts and examples.
Week 9 annotated companion	Supplies the numeric helper, access classes, shared-state classes, sensor hierarchy, composition example, and optional property implementation.
Original instructor guide / source map	Clarifies the intended domain distinctions, original optional scope, and the numerical-range caveat for changed pressure storage.
Worked additions	Resistor and voltage solutions, explicit repairs and regression checks, the reconstructed report defect, and 12 cumulative practice questions are labelled as solutions or extensions.

A.2. Clarifications that are not silent replacements

The absolute-pressure-only class and the finite-signed sensor hierarchy remain different models. Formatting examples retain their input assumptions. Ordinary attribute lookup is not expanded into a false universal rule for properties. The creation counter remains a standalone successful-initialization count rather than a live-object tracker. Pressure conversion can fail at extreme magnitudes without invalidating the original kPa observation. The `isinstance` nuance in Section 1.6 qualifies the original helper's docstring without changing its executable logic.

Source page 72 names a reporting-error example whose text is not printed there. Example 26 in this package is a labelled reconstruction of the described defect. The original `Sensor` implementation is retained. The local `low`/`high` placeholders on source page 66 are explained using the companion's actual `lower_limit`/`upper_limit` parameter names; they are not different class fields.

The optional properties, name mangling, and representations remain optional. No examination conditions, deadline, extra assignment, or certification claim is introduced. Supplementary official Python documentation is used only for narrow language-semantic qualifications; the course sequence remains the slide sequence.

A.3. Page-by-page source alignment

The first column is the page number in the 87-page source PDF, not its footer counter. The last column links to the relevant reading section and gives the page in these notes. Title, divider, reference, and closing pages are included.

PDF	Slide topic	Reading location
1	Title and course details	About these notes, p. 2
2	Week 9 at a glance	About these notes, p. 2
3	Course scope: Python classes and inheritance	About these notes, p. 2
4	Lecture 25 divider	Chapter 1, p. 6
5	Lecture 25 learning goals	Chapter 1, p. 6
6	Warm-up: a check can be bypassed	Section 1.1, p. 6
7	Write an invariant before choosing an interface	Section 1.2, p. 7
8	Hide the representation, not the meaning	Section 1.3, p. 8
9	Public and non-public names communicate intent	Section 1.4, p. 8
10	A getter returns the promised quantity	Section 1.5, p. 8
11	A setter checks before it changes state	Section 1.5, p. 8
12	Initialize through the same validated rule	Section 1.5, p. 8
13	A familiar helper supplies validation	Section 1.6, p. 9
14	Use the interface; inspect the returned result	Section 1.7, p. 11
15	Trace a rejected state change	Section 1.8, p. 12
16	Tests must cover both acceptance and rejection	Section 1.9, p. 12
17	Retain a regression check for state preservation	Section 1.9, p. 12
18	Printing inside a getter breaks reuse	Section 1.10, p. 13
19	A getter can accidentally expose a mutable list	Section 1.11, p. 14
20	Return a copy when a snapshot is promised	Section 1.12, p. 15
21	A stable interface can hide different storage	Section 1.13, p. 16
22	One client can use either representation	Section 1.13, p. 16
23	Write the public contract as a small table	Section 1.14, p. 17
24	Pair task: design controlled resistance access	Section 1.15, p. 18
25	Debugging clinic: inconsistent attribute names	Section 1.16, p. 19
26	Do not add accessors mechanically	Section 1.17, p. 20
27	An information-hiding checklist	Section 1.17, p. 20
28	Lecture 25 exit ticket	Section 1.18, p. 20
29	Lecture 26 divider	Chapter 2, p. 21
30	Lecture 26 learning goals	Chapter 2, p. 21
31	Warm-up: which data should be shared?	Section 2.1, p. 21
32	Define an ordinary class attribute	Section 2.2, p. 21
33	One shared default; two independent readings	Section 2.2, p. 21
34	Read the default through the class or instance	Section 2.3, p. 22
35	Change a common reporting preference	Section 2.3, p. 22
36	Instance assignment can shadow a class default	Section 2.4, p. 23
37	Trace the lookup, not just the last assignment	Section 2.4, p. 23
38	Ordinary attribute lookup: a useful local model	Section 2.5, p. 24
39	Choose the lookup policy inside a method	Section 2.6, p. 24
40	Prediction activity: find each attribute	Section 2.7, p. 25
41	A method is not inside the class local scope	Section 2.8, p. 26

Continued on the next page

PDF	Slide topic	Reading location
42	The shared-list trap	Section 2.9, p. 26
43	Two instances can refer to one class-level list	Section 2.9, p. 26
44	Create a new list during each initialization	Section 2.10, p. 27
45	Observe independent histories	Section 2.10, p. 27
46	Mutation and rebinding are different operations	Section 2.11, p. 28
47	A deliberate use of shared state: a counter	Section 2.12, p. 28
48	A failed creation should not count as success	Section 2.12, p. 28
49	Why self.total += 1 is not a class-wide count	Section 2.13, p. 29
50	Document what your counter does not measure	Section 2.14, p. 30
51	Test independence and control shared settings	Section 2.15, p. 31
52	Design clinic: a laboratory with ten sensors	Section 2.16, p. 32
53	Lecture 26 exit ticket	Section 2.17, p. 32
54	Lecture 27 divider	Chapter 3, p. 33
55	Lecture 27 learning goals	Chapter 3, p. 33
56	Warm-up: shared sensor responsibilities	Section 3.1, p. 33
57	Inheritance expresses a specialized kind of entity	Section 3.2, p. 33
58	The smallest child inherits existing behavior	Section 3.3, p. 34
59	Start with the parent contract	Section 3.4, p. 34
60	The parent establishes shared fields	Section 3.5, p. 35
61	One update rule can serve the whole hierarchy	Section 3.7, p. 37
62	A query uses the receiver, including a child	Section 3.8, p. 38
63	Specialize initialization with super()	Section 3.9, p. 38
64	Trace child construction step by step	Section 3.9, p. 38
65	Test conversions and unchanged state	Section 3.10, p. 40
66	A pressure child adds its own operation	Section 3.11, p. 40
67	Override a method without breaking its promise	Section 3.11, p. 40
68	Which report method runs?	Section 3.12, p. 41
69	Polymorphism: one loop, different implementations	Section 3.13, p. 42
70	Inspect exact type and inheritance separately	Section 3.14, p. 43
71	Class defaults can also be specialized by a child	Section 3.15, p. 43
72	A child must remain usable through the parent contract	Section 3.16, p. 44
73	Test inherited behavior as well as new methods	Section 3.17, p. 45
74	Debugging clinic: the parent was never initialized	Section 3.18, p. 46
75	Is-a is different from has-a	Section 3.19, p. 46
76	Design activity: add a VoltageSensor	Section 3.20, p. 47
77	Integrate and retain engineering evidence	Section 3.21, p. 48
78	Lecture 27 exit ticket	Section 3.22, p. 48
79	Optional: attribute-like access with a property	Section 4.1, p. 49
80	Optional: properties need separate backing data	Section 4.1, p. 49
81	Reference: the complete finite-number helper	Section 4.2, p. 50
82	Optional: see what name mangling actually does	Section 4.3, p. 51
83	Optional: readable and diagnostic text	Section 4.4, p. 51
84	Reference: interface and inheritance quick guide	Section 4.5, p. 52
85	Suggested reading and source basis	Section 4.6, p. 52
86	Week 9 synthesis and the next curriculum unit	Section 4.7, p. 53
87	Closing message: a reusable class keeps its promises	Section 4.7, p. 53

Bibliography

[1] Yanwei Wang. *Programming for Engineers: Week 9 — Classes, Information Hiding, and Inheritance*. Nazarbayev University, School of Engineering, Fall 2026. Supplied 87-page PDF, `week9_full(1).pdf`. Primary instructional basis.

[2] Yanwei Wang course-material package. *ENG 101 Week 9 Annotated Demo*, *Instructor Guide*, and *Source Map*. Earlier supplied Week 9 resources in this conversation. Used to recover model definitions and the original teaching-domain qualifications.

[3] Python Software Foundation. *Python Tutorial, Chapter 9: Classes*, especially class/instance variables, inheritance, and private-variable conventions. Official Python documentation. Supplementary language reference checked 15 September 2026.

[4] Python Software Foundation. *Built-in Functions*: `isinstance`, `issubclass`, `property`, and `super`. Official Python documentation. Supplementary language reference checked 15 September 2026.

Further reading reproduced as recommendations from source page 85: NU ENG 101 Course Specification, Session 9; Y. Wang, *Programming for Engineers with Python*, supplied manuscript v20260815, Sections 15.13 and 15.18–15.24, Appendix H.8–H.14 and H.22–H.27; J. V. Guttag, *Introduction to Computation and Programming Using Python*, second edition, Sections 8.2–8.3. These notes do not claim an independent comprehensive review of those larger sources.